

INTERNATIONAL ISLAMIC UNIVERSITY CHITTAGONG

Department of Computer Science and Engineering

CSE-2340 SOFTWARE DEVELOPMENT 1

Detailed Learnable Laboratory Manual

Course Information	Details
Semester	3rd Semester
Credit Hours	2
Contact Hours	4 laboratory classes per week
Type	Core, Engineering
Prerequisite	CSE-1222 (Computer Programming 2 Lab)
Co-requisite	CSE-1221 (Computer Programming 2)
Delivery	Laboratory / Practical
Course Structure	15 teaching weeks $\times$ 4 classes; Week 7 C3–C4 is one 100-minute mid-term practical block

1. How to Use This Manual

This manual follows the approved CSE-2340 syllabus in the same Week and C1, C2, C3, C4 order. C1 and C3 are teaching classes: the instructor introduces the concept, demonstrates the core pattern, and establishes correct terminology. C2 and C4 are application-focused practice or evaluation classes. Students should write code during the class, modify examples, inspect results, and debug their own work.

The manual is deliberately written so that a student can study the explanation, type the reference pattern, complete the guided laboratory, and continue with independent practice without needing a separate lecture note. External documentation is still useful for exact current syntax and deeper reference, but it is not a substitute for understanding the patterns presented here.

Core Learning Rule

Do not measure learning by how much code you can copy. Measure it by whether you can explain it, modify it, debug it, and rebuild the essential pattern.

2. Course Assessment

Assessment Component	Marks	Assessment Format
Attendance	10	Continuous
Mid-term In-lab Practical	20	100-minute individual practical assessment using Week 7 C3–C4
Six Mini Projects / Continuous Assessment	40	Individual projects with dedicated evaluation classes
Final Group Project / Showcase	30	Team project with individual viva
Total	100	

3. Standard Weekly Class Pattern

Class Role	Primary Purpose
C1 CLASS / TEACHING	Introduce the day's new concept, explain the mental model, demonstrate the core pattern, and establish correct terminology.
C2 PRACTICE	Students reproduce, modify, and troubleshoot the C1 concept through guided hands-on work.
C3 CLASS / TEACHING	Introduce the second major concept or connected concept for the week, with live demonstration.
C4 PRACTICE / EVALUATION	Students apply C3 and the week's combined skills; on project weeks, C4 becomes the dedicated individual evaluation.

Typical 50-Minute C1 / C3 Pattern

Time	Activity
0–5 min	Recall / motivation
5–15 min	Concept explanation
15–30 min	Instructor live build
30–40 min	Short student replication / checkpoint
40–47 min	Debug / questions / interpretation
47–50 min	Exit check

Typical 50-Minute C2 / C4 Practice Pattern

Time	Activity
0–5 min	Task briefing and success criteria
5–10 min	Instructor demonstrates the first step or constraint
10–35 min	Student guided practice
35–45 min	Debugging and individual support
45–50 min	Submission, demonstration, or exit check

FINAL CLASS-STRUCTURE CONTROL - READ BEFORE USING THIS MANUAL

CSE-2340 is not organized as four ordinary teaching classes each week.

C1 = CLASS / TEACHING. C2 = PRACTICE. C3 = CLASS / TEACHING. C4 = PRACTICE / EVALUATION.

C1 and C3 are the consolidated CLASS / TEACHING sessions. Concepts that previously appeared as teaching content in C2 or C4 are now introduced and demonstrated in C1 or C3. C2 and C4 are reserved for hands-on practice, application, debugging, evidence, or the dedicated project evaluation.

On mini-project weeks, C4 is the dedicated individual evaluation class. Week 7 C3–C4 is one 100-minute mid-term practical examination. Week 15 C4 is the separate final showcase slot.

Class	Role	Required student activity
C1	CLASS / TEACHING	Concept explanation, live build, core pattern, terminology and short replication.
C2	PRACTICE	Guided implementation, variation, debugging, DevTools inspection and evidence.
C3	CLASS / TEACHING	Second major/connected concept, live build and short replication.
C4	PRACTICE / EVALUATION	Combined implementation; dedicated individual evaluation on project weeks.

Practice-class rule: every C2 and C4 must contain hands-on work. These classes must not be converted into repeat lectures. The manual therefore provides separate practice objectives, guided tasks, independent variations, challenge/debugging work and exit checks.

Week 1: Web Foundations, Semantic HTML and Git

C1 - CLASS / TEACHING - Web Architecture and Development Environment: Client/Server, DNS, HTTP, Browser Rendering, Hosting, VS Code, Node/npm and Live Server

Why this matters: Understand what happens between entering a web address and seeing a page so that later front-end work is based on a correct mental model.

Learn This Before Coding

Web development is not only HTML and CSS. A browser is a client that requests resources from servers. DNS helps translate a domain name into an IP address. HTTP defines how requests and responses are exchanged. A response contains a status code, headers, and usually a body. The browser then parses HTML, discovers CSS, JavaScript, images and fonts, and constructs the page. Hosting is the process of making those resources available through a server.

Key Ideas

- Client: software that requests a resource.
- Server: software or infrastructure that responds to a request.
- DNS: translates domain names into IP addresses.
- HTTP method: describes the intended request operation.
- Status code: communicates the result category of a request.
- Rendering: the browser turns downloaded resources into a visible interface.

Reference Pattern

```
GET / HTTP/1.1
Host: example.com
Accept: text/html
```

HTTP/1.1 200 OK
Content-Type: text/html

Type the example yourself, run it, inspect the result, and then change at least one meaningful part. The goal is to understand the pattern rather than memorize the exact code.

Guided Laboratory

1. Open DevTools and the Network tab.
2. Reload a public page.
3. Identify the main document request.
4. Record its method, URL, status, response headers and timing.
5. Identify one CSS, JavaScript, image and font request.
6. Explain the order in which the browser obtains the resources.

Independent Practice

7. Find an example of a 200, 301 or 302, 404 and 500 status code.
8. Explain what each status communicates.
9. Draw a simple client → DNS → server → response → browser flow.

Challenge Task

Explain why a page can show HTML successfully while a CSS or image resource fails separately.

Common Errors and Diagnosis

- Confusing DNS with HTTP.
- Thinking a browser receives only one file.
- Treating every status code as an application error.
- Ignoring the Network tab when diagnosing resource problems.

Exit Check

10. What does DNS do?
11. What is contained in an HTTP response?
12. Why can one page produce many network requests?

Professional Habit

Save a working state, test it, and create a meaningful Git commit before leaving the class.

C2 - PRACTICE - Practice: Development Environment and First HTML Page

Why this matters: Prepare a repeatable local development environment and create the first working HTML page.

Learn This Before Coding

VS Code is the working environment used throughout the course. A browser can open an HTML file directly, but a local development server provides a predictable development environment and becomes increasingly important as projects use modules, APIs and build tools. Node.js provides a JavaScript runtime used by development tooling, while npm manages packages and scripts.

Key Ideas

- Editor: where source files are created and changed.
- Node.js: JavaScript runtime used by modern tooling.
- npm: package manager and script runner.

- Local server: serves the project through HTTP during development.
- Project folder: keeps related files and resources organized.

Reference Pattern

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>CSE-2340</title>
</head>
<body>
  <h1>CSE-2340</h1>
  <p>My first local web page.</p>
</body>
</html>
```

Type the example yourself, run it, inspect the result, and then change at least one meaningful part. The goal is to understand the pattern rather than memorize the exact code.

Guided Laboratory

13. Verify node --version and npm --version.
14. Create a clean course project folder.
15. Create index.html.
16. Open the project with Live Server.
17. Change the title and visible content.
18. Inspect the document request in DevTools.

Independent Practice

19. Create a second page.
20. Add an image and a link.
21. Change the viewport meta tag and explain why it matters.
22. Save a short README describing how to run the page.

Challenge Task

Explain why using a local server is useful even when the page appears to work by opening the HTML file directly.

Common Errors and Diagnosis

- Opening the wrong folder.
- Saving files with incorrect extensions.
- Forgetting the viewport meta tag.
- Not checking the browser Console and Network panels.

Exit Check

23. Why use a local server?
24. What does npm become useful for later?
25. Where should the project files live?

Professional Habit

Save a working state, test it, and create a meaningful Git commit before leaving the class.

Why this matters: Build meaningful HTML structure before styling so that the browser, assistive technologies and future CSS all have clear information to work with.

Learn This Before Coding

Semantic HTML uses elements according to their meaning rather than choosing tags only because they look a certain way. A page should normally have a clear document structure. Headings communicate hierarchy. Links navigate to another resource. Buttons perform actions. Images need useful alternative text when the image communicates information. Forms group user input and use labels so that controls are understandable.

Key Ideas

- Use one meaningful page heading and logical heading levels.
- Use header, nav, main, section, article, aside and footer when their meaning fits.
- Use anchor elements for navigation and button elements for actions.
- Associate labels with form controls.
- Use alt text that communicates the purpose of an informative image.

Reference Pattern

```
<main>
  <section aria-labelledby="courses-title">
    <h1 id="courses-title">My Courses</h1>
    <article>
      <h2>Software Development 1</h2>
      <p>Front-end development fundamentals.</p>
      <a href="details.html">View details</a>
    </article>
  </section>
</main>
```

Type the example yourself, run it, inspect the result, and then change at least one meaningful part. The goal is to understand the pattern rather than memorize the exact code.

Guided Laboratory

26. Create a department profile page.
27. Use one h1 and logical h2 headings.
28. Add nav, sections, article and footer.
29. Add an informative image with useful alt text.
30. Add a small registration form with labels.
31. Inspect the HTML tree in DevTools.

Independent Practice

32. Replace generic div elements with semantic elements where appropriate.
33. Test keyboard navigation.
34. Explain every major element in your page to a partner.

Challenge Task

Build a page whose HTML structure would still make sense if all CSS were removed.

Common Errors and Diagnosis

- Using headings only for font size.
- Using div instead of meaningful semantic elements everywhere.
- Using anchors as buttons.
- Missing form labels.
- Unhelpful alt text.

Exit Check

35. Why semantic HTML?
36. When should you use a button rather than an anchor?
37. Why should form controls be labeled?

Professional Habit

Save a working state, test it, and create a meaningful Git commit before leaving the class.

C4 - PRACTICE - Practice: Git Repository, Commit, Push and .gitignore Workflow

Why this matters: Start professional version control from the first project and understand the difference between local history and a remote repository.

Learn This Before Coding

Git records changes to a project. A working tree contains current files, staging selects changes for the next commit, and a commit records a snapshot in project history. A remote repository provides a shared online copy. The .gitignore file prevents files such as generated dependencies or local configuration from being accidentally committed.

Key Ideas

- git init creates local repository metadata.
- git status shows current changes.
- git add stages changes.
- git commit records a snapshot.
- git push sends commits to a remote repository.
- .gitignore defines files Git should ignore.

Reference Pattern

```
git init
git status
git add .
git commit -m "Create semantic homepage"
git branch -M main
git remote add origin <repository-url>
git push -u origin main
```

Type the example yourself, run it, inspect the result, and then change at least one meaningful part. The goal is to understand the pattern rather than memorize the exact code.

Guided Laboratory

38. Initialize the Week 1 project.
39. Create a .gitignore.
40. Create the GitHub repository.
41. Make the first commit.

42. Push the repository.
43. Change the page and create a second meaningful commit.

Independent Practice

44. Inspect git log --oneline.
45. Compare git status before and after staging.
46. Create a third focused commit rather than one huge unrelated commit.

Challenge Task

Explain add versus commit and commit versus push without using the words as if they were synonyms.

Common Errors and Diagnosis

- Committing generated folders.
- Using vague commit messages.
- Pushing without checking the repository.
- Thinking a remote repository replaces local Git history.

Exit Check

47. What is the difference between add and commit?
48. What is the difference between commit and push?
49. Why is .gitignore useful?

Professional Habit

Save a working state, test it, and create a meaningful Git commit before leaving the class.

Week 2: HTML Patterns, Accessibility and CSS Foundations

C1 - CLASS / TEACHING - Navigation, Forms, Tables and Accessibility: Landmarks, Alt Text, Labels, Heading Order, Keyboard Navigation, Focus and Contrast

Why this matters: Create practical HTML interfaces that use navigation, forms and tables for their intended purposes.

Learn This Before Coding

Navigation groups links that help users move through a site. Forms collect user input and should give each control a meaningful name, type and label. Tables are appropriate for genuinely tabular relationships such as schedules, marks or structured records. A table should have clear headings and a logical relationship between headings and data.

Key Ideas

- Use nav for navigation.
- Use labels and name attributes in forms.
- Choose input types that match the expected data.
- Use thead, tbody, th and td for tables.
- Do not use tables for general page layout.

Reference Pattern

```
<form>
  <label for="email">Email</label>
```



```

<input id="email" name="email" type="email">
<button type="submit">Send</button>
</form>

<table>
<thead>
<tr><th>Course</th><th>Credit</th></tr>
</thead>
<tbody>
<tr><td>CSE-2340</td><td>2</td></tr>
</tbody>
</table>

```

Type the example yourself, run it, inspect the result, and then change at least one meaningful part. The goal is to understand the pattern rather than memorize the exact code.

Guided Laboratory

50. Build a navigation bar with three links.
51. Build a registration form.
52. Create a course timetable table.
53. Use appropriate input types.
54. Test tab order.

Independent Practice

55. Add select, checkbox and radio controls.
56. Add a meaningful table caption.
57. Explain why a table is appropriate for a timetable but not for a page layout.

Challenge Task

Create a small student information page using navigation, a form and a table while keeping the document semantically structured.

Common Errors and Diagnosis

- Missing name attributes.
- Using the wrong input type.
- Using tables for layout.
- Leaving table headings unclear.

Exit Check

58. What does the name attribute do?
59. When should you use a table?
60. Why should form controls have labels?

Professional Habit

Save a working state, test it, and create a meaningful Git commit before leaving the class.

C2 - PRACTICE - Practice: Navigation, Forms, Tables and Accessibility Corrections

Why this matters: Convert the HTML concepts from C1 into a complete page and learn to modify an existing structure without breaking it.

Learn This Before Coding

Practice should move from copying a pattern to making controlled changes. Students should be able to identify the structural parts of a page, modify one part, test the result, and explain why the change is correct.

Key Ideas

- Separate structure from appearance.
- Keep navigation links meaningful.
- Keep forms readable and logically grouped.
- Use tables only when row-and-column relationships are meaningful.
- Test the page with keyboard navigation.

Reference Pattern

```
<header>
  <nav aria-label="Primary">
    <a href="index.html">Home</a>
    <a href="courses.html">Courses</a>
    <a href="contact.html">Contact</a>
  </nav>
</header>
```

Type the example yourself, run it, inspect the result, and then change at least one meaningful part. The goal is to understand the pattern rather than memorize the exact code.

Guided Laboratory

61. Build a complete course information page.
62. Add navigation.
63. Add a registration form.
64. Add a timetable table.
65. Validate the structure in DevTools.

Independent Practice

66. Create a second variation for a student club.
67. Replace one form field with a more suitable input type.
68. Change the table content without changing its structure.

Challenge Task

Given a page containing mostly div elements, refactor the structure into meaningful HTML without changing the visible content.

Common Errors and Diagnosis

- Changing content when the task only requires structural changes.
- Breaking links by changing filenames.
- Using headings in the wrong order.
- Forgetting to test form controls.

Exit Check

69. Can you explain the purpose of every major element in your page?
70. Can you modify the page without breaking its structure?

Professional Habit

Save a working state, test it, and create a meaningful Git commit before leaving the class.

Why this matters: Make front-end interfaces usable by a wider range of users by applying practical accessibility fundamentals.

Learn This Before Coding

Accessibility begins with correct HTML. Landmarks help users understand page regions. Alt text communicates the purpose of informative images. Labels identify controls. Heading order provides a navigable document outline. Keyboard users need visible focus and logical tab order. Text and interface elements need sufficient contrast.

Key Ideas

- Semantic landmarks reduce ambiguity.
- Alt text should communicate meaning, not describe irrelevant decoration.
- Every form control needs an accessible name.
- Headings should represent hierarchy.
- Interactive elements must be keyboard accessible.
- Focus should remain visible.
- Contrast should be checked rather than guessed.

Reference Pattern

```

```

```
<label for="search">Search</label>
```

```
<input id="search" name="search" type="search">
```

```
<button type="button">Open menu</button>
```

Type the example yourself, run it, inspect the result, and then change at least one meaningful part. The goal is to understand the pattern rather than memorize the exact code.

Guided Laboratory

71. Inspect a deliberately flawed page.
72. Tab through every control.
73. Find unlabeled inputs.
74. Check heading order.
75. Inspect image alt text.
76. Check visible focus.
77. Use browser tools to inspect contrast where available.

Independent Practice

78. Correct at least five accessibility issues.
79. Explain why each correction matters.
80. Repeat the keyboard test after correction.

Challenge Task

Perform an accessibility audit without changing visual content unnecessarily. Prioritize structural and interaction problems first.

Common Errors and Diagnosis

- Removing focus outlines without replacement.
- Using placeholder text as a substitute for labels.
- Skipping heading levels for visual size.
- Writing alt text such as 'image' without useful information.

Exit Check

81. Why are landmarks useful?
82. Why must labels be associated with controls?
83. What makes a focus indicator important?

Professional Habit

Save a working state, test it, and create a meaningful Git commit before leaving the class.

C4 - PRACTICE - Practice: CSS Box Model, Display, Backgrounds and Layout Debugging

Why this matters: Apply the C3 accessibility checklist to an existing page and document evidence of each correction.

Learn This Before Coding

An audit is more useful when it is evidence-driven. Students should identify a specific issue, explain the user impact, apply the smallest appropriate correction, and retest.

Key Ideas

- Audit structure first.
- Audit keyboard interaction.
- Audit form labels.
- Audit images.
- Audit headings.
- Audit focus and contrast.
- Retest after each meaningful correction.

Reference Pattern

Issue: input has no accessible label

Correction:

```
<label for="name">Name</label>
```

```
<input id="name" name="name">
```

Type the example yourself, run it, inspect the result, and then change at least one meaningful part. The goal is to understand the pattern rather than memorize the exact code.

Guided Laboratory

84. Open the provided flawed page.
85. Record at least six issues.
86. Correct the issues.
87. Test the page again.
88. Prepare a short issue → fix → verification list.

Independent Practice

89. Give a partner the corrected page.
90. Ask the partner to find one remaining accessibility problem.

91. Fix the discovered issue and record it.

Challenge Task

Complete an accessibility audit using only observable evidence from the page and browser tools.

Common Errors and Diagnosis

- Fixing only visual appearance.
- Changing many unrelated files.
- Failing to retest after correction.
- Treating accessibility as a single checkbox.

Exit Check

92. What was the most important issue you found?

93. What evidence proved that your correction worked?

Professional Habit

Save a working state, test it, and create a meaningful Git commit before leaving the class.

Week 3: Flexbox, Grid, Responsive Design and Mini Project 1

C1 - CLASS / TEACHING - Flexbox, CSS Grid and Responsive/Mobile-First Design

Why this matters: Build reliable one-dimensional layouts using the Flexbox layout model.

Learn This Before Coding

Flexbox arranges items along a main axis and a cross axis. The parent becomes a flex container with `display:flex`. `flex-direction` chooses the main axis. `justify-content` controls distribution along the main axis, while `align-items` controls alignment on the cross axis. `gap` creates consistent spacing between items.

Key Ideas

- Container properties control the layout of children.
- `row` is the default direction.
- `justify-content` works on the main axis.
- `align-items` works on the cross axis.
- `flex-wrap` allows items to move to another line.
- `gap` is preferable to many ad-hoc margins for consistent spacing.

Reference Pattern

```
.nav { display:flex; align-items:center; justify-content:space-between; gap:1rem; }  
.cards { display:flex; flex-wrap:wrap; gap:1rem; }  
.card { flex:1 1 16rem; }
```

Type the example yourself, run it, inspect the result, and then change at least one meaningful part. The goal is to understand the pattern rather than memorize the exact code.

Guided Laboratory

94. Build a horizontal navigation.

95. Build a wrapping card row.

96. Change flex-direction and predict the result.
97. Use gap for spacing.
98. Inspect computed flex properties.

Independent Practice

99. Create a responsive header.
100. Create a row that wraps naturally.
101. Build a two-column section that becomes a vertical stack.

Challenge Task

Before using positioning, solve the layout with Flexbox wherever the problem is primarily one-dimensional.

Common Errors and Diagnosis

- Confusing main and cross axes.
- Using fixed widths that prevent wrapping.
- Adding margins everywhere instead of using gap.
- Using absolute positioning for ordinary layout.

Exit Check

102. What determines the main axis?
103. Where does justify-content operate?
104. When is flex-wrap useful?

Professional Habit

Save a working state, test it, and create a meaningful Git commit before leaving the class.

C2 - PRACTICE - Practice: Responsive Flexbox/Grid Layouts and DevTools Inspection

Why this matters: Translate a written layout requirement into a working Flexbox implementation and practice predicting layout behavior.

Learn This Before Coding

Good CSS practice starts from requirements rather than random declarations. Students should identify the layout relationship first, choose the appropriate Flexbox properties, implement, inspect, and then test at different widths.

Key Ideas

- Identify parent-child relationships.
- Choose row or column deliberately.
- Use alignment and spacing properties intentionally.
- Use wrapping when the design allows it.
- Test narrow and wide layouts.

Reference Pattern

```
<section class="cards">
  <article class="card">Course 1</article>
  <article class="card">Course 2</article>
  <article class="card">Course 3</article>
</section>
```

Type the example yourself, run it, inspect the result, and then change at least one meaningful part. The goal is to understand the pattern rather than memorize the exact code.

Guided Laboratory

105. Read the prepared design requirements.
106. Build the header with Flexbox.
107. Build the card section.
108. Make the cards wrap.
109. Test at narrow and wide viewport sizes.

Independent Practice

110. Recreate a second layout from written requirements.
111. Change one requirement and identify the minimum CSS change.
112. Use DevTools to inspect the flex container.

Challenge Task

Given a layout description rather than CSS, explain which Flexbox properties you choose and why before writing the code.

Common Errors and Diagnosis

- Choosing properties by trial and error.
- Forgetting which axis a property controls.
- Using fixed dimensions unnecessarily.
- Testing only one viewport.

Exit Check

113. Which requirement determines flex-direction?
114. How did you verify that your layout is responsive?

Professional Habit

Save a working state, test it, and create a meaningful Git commit before leaving the class.

C3 - CLASS / TEACHING - Specificity, Cascade, Pseudo-Classes and Practical DevTools Inspection; Live Build of a Complete Static Page

Why this matters: Use Grid for two-dimensional layouts and understand the CSS rules that determine which declaration wins.

Learn This Before Coding

CSS Grid is useful when rows and columns both matter. Mobile-first design starts with a simple narrow-screen layout and adds rules for wider screens. Specificity, source order and importance influence which declaration applies. Pseudo-classes describe states such as hover and focus. DevTools lets you inspect computed styles and identify overridden rules.

Key Ideas

- Grid defines rows and columns.
- Mobile-first CSS starts with the base layout and adds larger-screen rules.
- Specificity determines selector strength.
- Later rules can win when specificity is equal.
- Pseudo-classes represent interaction or structural states.

- Computed styles show the final applied values.

Reference Pattern

```
.grid { display:grid; grid-template-columns:1fr; gap:1rem; }
@media (min-width:48rem) {
  .grid { grid-template-columns:repeat(3,1fr); }
}
button:hover { text-decoration:underline; }
button:focus-visible { outline:2px solid currentColor; }
```

Type the example yourself, run it, inspect the result, and then change at least one meaningful part. The goal is to understand the pattern rather than memorize the exact code.

Guided Laboratory

115. Create a one-column mobile grid.
116. Add three columns at a wider breakpoint.
117. Test 320px, 768px and 1200px.
118. Create a specificity conflict.
119. Use DevTools to determine which declaration wins.
120. Add hover and focus states.

Independent Practice

121. Build a responsive course dashboard.
122. Create one intentional specificity conflict and diagnose it.
123. Inspect computed styles for a spacing problem.

Challenge Task

Given a CSS declaration that is not taking effect, diagnose whether the problem is selector matching, specificity, source order, or another rule before changing code.

Common Errors and Diagnosis

- Using media queries without a mobile base.
- Assuming the last rule always wins.
- Ignoring selector specificity.
- Removing focus styles without replacement.
- Checking only the source CSS instead of computed styles.

Exit Check

124. Why is Grid useful here?
125. What is mobile-first design?
126. How do you diagnose an overridden CSS declaration?

Professional Habit

Save a working state, test it, and create a meaningful Git commit before leaving the class.

C4 - PRACTICE / EVALUATION - Mini Project 1 Evaluation: Semantic HTML/CSS Responsive Page and Deployment

PRACTICE / EVALUATION CLASS: Students apply C3 and combined weekly skills. On mini-project weeks, C4 is the dedicated individual evaluation with live verification.

Why this matters: Verify that each student can independently build and deploy a small responsive front-end page using the first three weeks of material.

Learn This Before Coding

MP1 is an individual evaluation. The expected artifact uses semantic HTML, CSS layout, responsive behavior and deployment. The evaluation is evidence-based: the student demonstrates the running project and explains the implementation.

Key Ideas

- Semantic structure must be meaningful.
- CSS must control layout rather than malformed HTML.
- The interface must behave appropriately at tested widths.
- The deployed page must be accessible.
- The student must be able to explain and make a small modification.

Reference Pattern

Suggested evidence:

- Repository
- Deployed URL
- Semantic page structure
- Responsive layout
- Individual explanation

Type the example yourself, run it, inspect the result, and then change at least one meaningful part. The goal is to understand the pattern rather than memorize the exact code.

Guided Laboratory

127. Check repository and deployed URL.
128. Call students in a fixed order.
129. Ask each student for a short live demonstration.
130. Ask one or two ownership questions.
131. If time permits, request one small live modification.
132. Record the rubric result immediately.

Independent Practice

133. Before the evaluation, test 320px, tablet and desktop widths.
134. Verify links and images.
135. Review Git history.
136. Prepare a two-minute explanation of the page structure and layout decisions.

Challenge Task

Make one small live modification requested by the instructor without breaking the deployed application.

Common Errors and Diagnosis

- Presentation-only evidence.
- Student cannot explain semantic choices.
- Layout works only at one width.
- Deployment is broken.
- Large uncommitted changes make ownership unclear.

Exit Check

137. What is the semantic structure of your page?
138. Which layout problem did Flexbox or Grid solve?
139. What would you change if the viewport became narrower?

Professional Habit

Save a working state, test it, and create a meaningful Git commit before leaving the class.

Week 4: Design Handoff, CSS Positioning, z-index and Tailwind CSS v4

C1 - CLASS / TEACHING - Design Handoff Using Figma or Pixso and CSS Positioning: Relative, Absolute, Fixed, Sticky and z-index

Why this matters: Learn to read a design handoff as implementation evidence rather than treating a design file as a picture to copy blindly.

Learn This Before Coding

A design handoff communicates layout, spacing, typography, dimensions and assets. Frames show regions of a design. Measurements help identify spacing relationships. Exportable assets should be used at appropriate dimensions and formats. The developer's task is to translate design intent into semantic structure and responsive rules.

Key Ideas

- Frame: a defined design region.
- Spacing: distance between elements or sections.
- Measurement: evidence about size or position.
- Asset export: obtaining the required image or graphic resource.
- Design intent: what the interface is trying to communicate, not only its exact pixels.

Reference Pattern

Implementation note:

Design measurement → semantic structure → CSS/Tailwind rules

Do not begin by absolutely positioning every element.

Type the example yourself, run it, inspect the result, and then change at least one meaningful part. The goal is to understand the pattern rather than memorize the exact code.

Guided Laboratory

140. Open the prepared Figma or Pixso file.
141. Identify page regions.
142. Record key spacing and dimensions.
143. Identify typography hierarchy.
144. Identify assets that need export.
145. Create an implementation checklist.

Independent Practice

146. Reproduce one selected section.
147. Compare the implementation with the design.
148. Record three differences and decide which are important versus cosmetic.

Challenge Task

Translate one design section into semantic HTML plus layout rules before writing detailed styling.

Common Errors and Diagnosis

- Copying coordinates instead of relationships.
- Ignoring responsive behavior.
- Exporting assets at inappropriate sizes.
- Using absolute positioning for ordinary layout.

Exit Check

149. What information does a design handoff provide?
150. Why should relationships matter more than isolated pixel values?

Professional Habit

Save a working state, test it, and create a meaningful Git commit before leaving the class.

C2 - PRACTICE - Practice: Design Handoff Translation and Positioning Requirements

Why this matters: Practice translating a visual design specification into a responsive implementation while preserving semantic structure.

Learn This Before Coding

The implementation should start with page regions, then layout, then typography, spacing and assets. A design at one viewport does not automatically define behavior at every viewport, so students must make reasonable responsive decisions within the provided requirements.

Key Ideas

- Read before coding.
- Identify repeated patterns.
- Separate layout decisions from visual details.
- Use semantic HTML.
- Test the result against the design and at a narrower width.

Reference Pattern

```
<section class="hero">
  <div class="hero-content">
    <h1>Build with structure.</h1>
    <p>Responsive interfaces start with clear relationships.</p>
  </div>
</section>
```

Type the example yourself, run it, inspect the result, and then change at least one meaningful part. The goal is to understand the pattern rather than memorize the exact code.

Guided Laboratory

151. Inspect the prepared design.
152. Create semantic regions.
153. Implement the selected section.
154. Use measured spacing as a guide.
155. Export and use the required asset.
156. Test at the target width and a narrower width.

Independent Practice

157. Reproduce a second section.
158. Replace one fixed measurement with a responsive rule.
159. Explain one deliberate deviation from the reference.

Challenge Task

Given a design screenshot, explain the implementation hierarchy before writing CSS.

Common Errors and Diagnosis

- Starting with decoration instead of structure.
- Ignoring the design's spacing system.
- Breaking the layout at smaller widths.
- Using unnecessary absolute positioning.

Exit Check

160. What should you identify before writing detailed CSS?
161. How did you make the design responsive?

Professional Habit

Save a working state, test it, and create a meaningful Git commit before leaving the class.

C3 - CLASS / TEACHING - Tailwind CSS v4: Setup, Typography, Spacing and Responsive Flex/Grid; Live Build of a Responsive Tailwind Page

Why this matters: Use positioning only when the visual relationship actually requires it, and learn the practical subset of Tailwind CSS v4 used in this course.

Learn This Before Coding

Normal document flow should solve ordinary layout. `position: relative` establishes a positioning context. `absolute` removes an element from normal flow and positions it relative to an appropriate containing block. `fixed` attaches an element to the viewport. `sticky` behaves like a relatively positioned element until a scroll threshold is reached. `z-index` controls stacking order within relevant stacking contexts. Tailwind expresses common CSS decisions through utility classes.

Key Ideas

- Relative is often used to establish a containing block.
- Absolute is useful for overlays, badges and anchored decorative elements.
- Fixed is useful for viewport-attached controls.
- Sticky is useful for headers or side regions that should remain visible during scrolling.
- `z-index` should solve a real stacking relationship.
- Tailwind utilities should describe the same layout concepts learned in CSS.

Reference Pattern

```
<div class="relative">
  <span class="absolute right-2 top-2">New</span>
</div>

<div class="flex flex-col gap-4 md:flex-row">
  <article class="flex-1">A</article>
  <article class="flex-1">B</article>
</div>
```

Type the example yourself, run it, inspect the result, and then change at least one meaningful part. The goal is to understand the pattern rather than memorize the exact code.

Guided Laboratory

162. Create a relative container.
163. Add an absolute badge.
164. Create a sticky section header.
165. Demonstrate a fixed control.
166. Create a Tailwind project using the course setup.
167. Build a responsive flex/grid section with Tailwind utilities.

Independent Practice

168. Recreate a small card from the design handoff.
169. Convert a small CSS section into Tailwind utilities.
170. Inspect the generated layout and explain each major utility group.

Challenge Task

Build an interface where one element is intentionally overlaid and explain why absolute positioning is appropriate for that element but not for the whole page.

Common Errors and Diagnosis

- Using absolute positioning for page layout.
- Using z-index values without understanding the stacking relationship.
- Mixing utility classes randomly.
- Forgetting responsive variants.

Exit Check

171. When is absolute positioning appropriate?
172. What problem does relative positioning often solve?
173. How does Tailwind represent responsive layout changes?

Professional Habit

Save a working state, test it, and create a meaningful Git commit before leaving the class.

C4 - PRACTICE - Practice: Responsive Tailwind Page from Design/Reference

Why this matters: Turn the week's design and Tailwind concepts into a complete responsive page.

Learn This Before Coding

The goal is implementation fluency. Students should be able to read a reference, build the structure, use Tailwind for typography and spacing, create responsive Flexbox/Grid behavior, and verify the result.

Key Ideas

- Structure first.
- Use Tailwind utilities consistently.
- Prefer responsive variants over duplicated markup.
- Keep the design understandable at multiple widths.
- Verify the deployed-ready build locally.

Reference Pattern

```

<main class="mx-auto w-[92%] max-w-6xl">
  <section class="grid gap-6 md:grid-cols-2">
    <div>Content</div>
    <div>Visual</div>
  </section>
</main>

```

Type the example yourself, run it, inspect the result, and then change at least one meaningful part. The goal is to understand the pattern rather than memorize the exact code.

Guided Laboratory

174. Create the project from the supplied reference.
175. Build the page structure.
176. Apply typography and spacing.
177. Implement responsive layout.
178. Test narrow, tablet and desktop widths.

Independent Practice

179. Add a second responsive section.
180. Replace one arbitrary value with a more systematic utility.
181. Perform a short peer review.

Challenge Task

Rebuild one section after deleting your CSS/Tailwind declarations and explain the layout decisions from memory.

Common Errors and Diagnosis

- Overusing arbitrary values.
- Ignoring semantic HTML.
- Designing only for desktop.
- Using utility classes without understanding the underlying CSS.

Exit Check

182. Can you explain the main Tailwind classes you used?
183. Does the page remain usable on a narrow screen?

Professional Habit

Save a working state, test it, and create a meaningful Git commit before leaving the class.

Week 5: Deployment, Image Optimization, JavaScript Fundamentals and Mini Project 2

C1 - CLASS / TEACHING - Responsive Deployment, Image Sourcing, Basic Image Optimization and JavaScript Fundamentals: Variables, Primitives and Template Literals

Why this matters: Treat deployment and resource cost as part of front-end engineering rather than as an afterthought.

Learn This Before Coding

A working local page is not complete until the deployed result works. Image files can dominate page size, so students should choose appropriate dimensions and formats, avoid unnecessary resolution, and verify the deployed page rather than assuming local success means deployment success.

Key Ideas

- Deployment publishes the application so users can access it.
- Use appropriate image dimensions.
- Avoid unnecessarily large assets.
- Use meaningful filenames and alt text.
- Check the deployed page for broken paths and missing assets.

Reference Pattern

Responsive image practice:
``

Type the example yourself, run it, inspect the result, and then change at least one meaningful part. The goal is to understand the pattern rather than memorize the exact code.

Guided Laboratory

184. Choose images for a small page.
185. Check their dimensions and file sizes.
186. Deploy the page using the course-approved workflow.
187. Open the deployed URL.
188. Check images, CSS and navigation.
189. Inspect Network for unexpectedly large resources.

Independent Practice

190. Replace one oversized image with a more appropriate asset.
191. Record before/after file size.
192. Test the page on a slower connection if the browser allows it.

Challenge Task

Diagnose a deployed page where HTML loads but an image or stylesheet does not.

Common Errors and Diagnosis

- Using local absolute paths.
- Deploying without testing the production URL.
- Uploading unnecessarily huge images.
- Assuming an asset path that works locally will always work after deployment.

Exit Check

193. Why does image size matter?
194. What should you verify after deployment?
195. Where would you look if an image fails to load?

Professional Habit

Save a working state, test it, and create a meaningful Git commit before leaving the class.

C2 - PRACTICE - Practice: Deployment, Image Optimization and JavaScript Fundamentals

Why this matters: Perform the complete local-to-deployed workflow and verify that the published page behaves like the tested local version.

Learn This Before Coding

Deployment practice should include building, publishing, opening the public URL, checking assets, testing responsiveness and fixing any production-specific problem. The deployed application is the artifact users actually receive.

Key Ideas

- Build before publishing.
- Verify the public URL.
- Check relative asset paths.
- Test navigation.
- Test responsive behavior.
- Use Network and Console when something fails.

Reference Pattern

Verification checklist:

1. Open deployed URL.
2. Reload.
3. Test navigation.
4. Test images.
5. Test responsive layout.
6. Check Console.
7. Inspect Network.

Type the example yourself, run it, inspect the result, and then change at least one meaningful part. The goal is to understand the pattern rather than memorize the exact code.

Guided Laboratory

196. Deploy the prepared responsive page.
197. Test all navigation.
198. Test all images.
199. Open DevTools on the deployed page.
200. Fix one deployment issue if found.

Independent Practice

201. Create a deployment checklist.
202. Compare local and deployed behavior.
203. Document one issue and its root cause.

Challenge Task

Take a page that works locally, deploy it, and prove that the production version works with evidence rather than simply stating that it works.

Common Errors and Diagnosis

- Checking only localhost.
- Ignoring Console errors.
- Testing only the home page.

- Forgetting that deployment can expose incorrect paths.

Exit Check

- 204. What is the difference between local success and deployment success?
- 205. What evidence proves your deployment works?

Professional Habit

Save a working state, test it, and create a meaningful Git commit before leaving the class.

C3 - CLASS / TEACHING - JavaScript Data Handling: Arrays, Objects and Functions

Why this matters: Build the JavaScript foundation required for interactive front-end applications.

Learn This Before Coding

JavaScript stores data in variables and combines values through expressions. Primitive values include strings, numbers, booleans, null and undefined. Arrays represent ordered collections. Objects represent related properties. Functions package reusable behavior. Template literals make it convenient to construct strings with embedded values.

Key Ideas

- Use const by default and let when reassignment is required.
- Know the difference between primitive values and collections.
- Use arrays for ordered collections.
- Use objects for related properties.
- Use functions to express reusable operations.
- Use template literals for readable dynamic strings.

Reference Pattern

```
const studentName = "Mahir";
const credits = 2;
const course = {
  code: "CSE-2340",
  title: "Software Development 1"
};

const message = `${course.code}: ${course.title}`;

function add(a, b) {
  return a + b;
}

const values = [2, 4, 6];
```

Type the example yourself, run it, inspect the result, and then change at least one meaningful part. The goal is to understand the pattern rather than memorize the exact code.

Guided Laboratory

- 206. Create variables for a course.
- 207. Create an array of courses.
- 208. Create course objects.
- 209. Write a function that calculates a simple value.

210. Use a template literal to display course information.

Independent Practice

211. Create a student object containing nested information.

212. Write a function that accepts an object.

213. Modify one array item and observe the result.

214. Explain which values are primitives and which are objects.

Challenge Task

Given a small dataset, choose whether each piece of information should be a variable, array, object or function and explain the choice.

Common Errors and Diagnosis

- Using let for everything.
- Confusing an object with an array.
- Forgetting that functions can return values.
- Building long strings through repeated concatenation when a template literal is clearer.

Exit Check

215. What is the purpose of const?

216. When is an array appropriate?

217. When is an object appropriate?

218. Why use functions?

Professional Habit

Save a working state, test it, and create a meaningful Git commit before leaving the class.

C4 - PRACTICE / EVALUATION - Mini Project 2 Evaluation: Own Responsive Design Using Tailwind, Deployed

PRACTICE / EVALUATION CLASS: Students apply C3 and combined weekly skills. On mini-project weeks, C4 is the dedicated individual evaluation with live verification.

Why this matters: Verify that students can independently translate their own design idea into a responsive Tailwind implementation and deploy it.

Learn This Before Coding

The project should demonstrate the design-handoff, responsive layout, Tailwind and deployment skills taught through Week 5. The evaluation is individual and requires a running artifact that the student can explain.

Key Ideas

- The design should be the student's own implementation.
- Tailwind should be used meaningfully.
- The page should respond appropriately to different widths.
- The deployed URL should work.
- The student must explain implementation decisions.

Reference Pattern

Evidence:

- Repository
- Deployed URL
- Responsive page

- Tailwind implementation
- Individual explanation

Type the example yourself, run it, inspect the result, and then change at least one meaningful part. The goal is to understand the pattern rather than memorize the exact code.

Guided Laboratory

219. Check the repository and deployed URL.
220. Ask for a short live demonstration.
221. Inspect responsive behavior.
222. Ask one or two implementation questions.
223. Request one small live change when possible.
224. Record the result immediately.

Independent Practice

225. Test three viewport sizes.
226. Check images and links.
227. Review Git history.
228. Prepare a short explanation of the responsive strategy.

Challenge Task

Make a small live design change requested by the instructor and preserve responsive behavior.

Common Errors and Diagnosis

- Page only works at one width.
- Student cannot explain Tailwind classes.
- Deployment is broken.
- The project is mostly copied without understanding.

Exit Check

229. What responsive strategy did you use?
230. Which Tailwind utilities control the main layout?
231. How did you verify deployment?

Professional Habit

Save a working state, test it, and create a meaningful Git commit before leaving the class.

Week 6: DOM, Events and Interactive JavaScript

C1 - CLASS / TEACHING - DOM Manipulation: Query Selectors, Content, Attributes, classList, createElement and appendChild

Why this matters: Understand the DOM as the browser's object representation of the document and select elements reliably.

Learn This Before Coding

The DOM represents the document as a tree of nodes. JavaScript can select nodes, read properties, change content and attributes, and create new elements. `querySelector` returns the first matching element, while `querySelectorAll` returns a collection of matches.

Key Ideas

- DOM is a tree representation of the document.
- Selectors connect JavaScript to HTML elements.
- `textContent` changes text safely.
- Attributes can be read and changed.
- Always consider whether a selector can return null.

Reference Pattern

```
const title = document.querySelector("#page-title");
title.textContent = "Updated by JavaScript";

const links = document.querySelectorAll("nav a");
links.forEach(link => console.log(link.textContent));
```

Type the example yourself, run it, inspect the result, and then change at least one meaningful part. The goal is to understand the pattern rather than memorize the exact code.

Guided Laboratory

232. Create a page with headings, buttons and cards.
233. Select an element by id.
234. Select a group using a CSS selector.
235. Change text content.
236. Read an attribute.
237. Inspect the DOM in DevTools.

Independent Practice

238. Select several cards and change one property.
239. Add a missing-element check.
240. Explain the difference between one selected element and a collection of matches.

Challenge Task

Build a small script that can locate the correct UI element even after the visible text changes.

Common Errors and Diagnosis

- Incorrect selector.
- Selecting an element before the script runs.
- Assuming `querySelectorAll` returns one element.
- Ignoring null results.

Exit Check

241. What does `querySelector` return?
242. What does `querySelectorAll` return?
243. What is the DOM?

Professional Habit

Save a working state, test it, and create a meaningful Git commit before leaving the class.

C2 - PRACTICE - Practice: DOM Manipulation and Dynamic Element Creation

Why this matters: Build and modify DOM structures programmatically rather than editing HTML manually for every change.

Learn This Before Coding

Interactive applications often create UI from data. `classList` manages CSS classes. `createElement` creates new nodes. `appendChild` inserts a node into the document. Keeping data and rendering logic separate makes later React concepts easier to understand.

Key Ideas

- Use `textContent` for text.
- Use `setAttribute` or direct properties for attributes where appropriate.
- Use `classList` to manage classes.
- Use `createElement` to create nodes.
- Use `appendChild` to build a DOM tree.
- Keep source data separate from rendering code.

Reference Pattern

```
const card = document.createElement("article");
card.classList.add("card");

const title = document.createElement("h2");
title.textContent = "CSE-2340";

card.appendChild(title);
document.querySelector("#courses").appendChild(card);
```

Type the example yourself, run it, inspect the result, and then change at least one meaningful part. The goal is to understand the pattern rather than memorize the exact code.

Guided Laboratory

244. Create a container.
245. Create one card with JavaScript.
246. Add a heading and paragraph.
247. Add a class.
248. Append the card.
249. Create several cards from an array.

Independent Practice

250. Add an image and link to each generated card.
251. Add a delete button.
252. Clear and rerender the container.

Challenge Task

Create a render function that receives data and produces the complete UI without hard-coding repeated cards in HTML.

Common Errors and Diagnosis

- Using `innerHTML` without understanding the trade-off.
- Appending to the wrong container.
- Creating elements but never attaching them.
- Mixing data creation with DOM rendering.

Exit Check

- 253. What does createElement do?
- 254. Why use classList?
- 255. Why is a render function useful?

Professional Habit

Save a working state, test it, and create a meaningful Git commit before leaving the class.

C3 - CLASS / TEACHING - JavaScript Events: addEventListener, preventDefault and Practical Event Handling

Why this matters: Connect user actions to JavaScript behavior through event listeners.

Learn This Before Coding

Events represent things that happen in the browser, such as clicks, input, submission and keyboard actions. addEventListener registers a function for an event. The event object contains information about what happened. preventDefault stops a browser's default behavior when that behavior is not appropriate for the application.

Key Ideas

- Choose the correct event type.
- Use addEventListener rather than mixing behavior into markup unnecessarily.
- Read event.target when appropriate.
- Use preventDefault for form submission when the page should not reload.
- Separate event handling from rendering.

Reference Pattern

```
const form = document.querySelector("#course-form");

form.addEventListener("submit", event => {
  event.preventDefault();
  console.log("Submitted");
});
```

Type the example yourself, run it, inspect the result, and then change at least one meaningful part. The goal is to understand the pattern rather than memorize the exact code.

Guided Laboratory

- 256. Create a submit handler.
- 257. Prevent page reload.
- 258. Read input values.
- 259. Display feedback.
- 260. Add a button interaction.
- 261. Add a live input event.

Independent Practice

- 262. Build a three-field registration form.
- 263. Add validation feedback.
- 264. Add a live character counter.
- 265. Explain the event → handler → state/data → UI sequence.

Challenge Task

Create a small interaction where one user action changes the visible interface and explain every step in the flow.

Common Errors and Diagnosis

- Wrong event type.
- Handler not firing because the selector is wrong.
- Form reloads because preventDefault was omitted.
- Using the wrong target element.

Exit Check

266. What is event.target?
267. Why is preventDefault useful?
268. Which event is suitable for live input?

Professional Habit

Save a working state, test it, and create a meaningful Git commit before leaving the class.

C4 - PRACTICE - Practice: Interactive DOM Application and Debugging

Why this matters: Combine DOM selection, rendering, events and debugging into a small application that can be explained and modified.

Learn This Before Coding

A small DOM application can be modeled as data plus actions plus rendering. User actions trigger events. Events change data or request a change. Rendering reflects the current data. Debugging should follow this flow rather than relying on random edits.

Key Ideas

- Separate application data from DOM rendering.
- Create a render function.
- Use events to trigger actions.
- Update data intentionally.
- Rerender when the visible state changes.
- Debug one cause at a time.

Reference Pattern

```
let tasks = [
  { id: 1, title: "Learn DOM", done: false },
  { id: 2, title: "Practice events", done: true }
];

function render() {
  const list = document.querySelector("#task-list");
  list.innerHTML = "";
  tasks.forEach(task => {
    const li = document.createElement("li");
    li.textContent = task.title;
    list.appendChild(li);
  });
}
```

Type the example yourself, run it, inspect the result, and then change at least one meaningful part. The goal is to understand the pattern rather than memorize the exact code.

Guided Laboratory

- 269. Build a task, expense or product application.
- 270. Create a render function.
- 271. Add create/delete/toggle behavior.
- 272. Introduce one small bug.
- 273. Use Console and Elements to diagnose it.
- 274. Commit the working version.

Independent Practice

- 275. Add an empty state.
- 276. Add an item count.
- 277. Add a completed filter.
- 278. Have a partner introduce one bug and diagnose it using evidence.

Challenge Task

Explain the complete user action → event → data change → render flow for your application.

Common Errors and Diagnosis

- UI changes but data does not.
- Data changes but render is not called.
- Null DOM reference.
- Changing many files instead of isolating the problem.

Exit Check

- 279. What is application state in this DOM application?
- 280. What is the first step when debugging a visible failure?

Professional Habit

Save a working state, test it, and create a meaningful Git commit before leaving the class.

Week 7: Array Methods, Debugging and Mid-term Practical Examination

C1 - CLASS / TEACHING - Array Methods for UI Work: map, filter, forEach and find; DOM Build and Debug Preparation

Why this matters: Transform data into UI-ready collections using the array methods most frequently needed in front-end development.

Learn This Before Coding

map transforms every element and returns a new array. filter returns the elements that satisfy a condition. find returns the first matching element. forEach performs an action for each element but does not create a transformed array.

Key Ideas

- map changes the representation of every item.
- filter selects zero or more items.
- find selects at most one item.
- forEach is mainly for side effects.
- These methods normally do not mutate the source array.

Reference Pattern

```
const hardware = products.filter(p => p.category === "hardware");
const names = products.map(p => p.name);
const selected = products.find(p => p.id === 2);
```

Type the example yourself, run it, inspect the result, and then change at least one meaningful part. The goal is to understand the pattern rather than memorize the exact code.

Guided Laboratory

281. Create product data.
282. Filter by category.
283. Map products to names.
284. Find one product.
285. Render the results.
286. Test an empty result.

Independent Practice

287. Combine filter and map.
288. Write one example for each method.
289. Create a searchable/filterable dataset ready for DOM rendering.

Challenge Task

Given a UI requirement, decide whether you need map, filter, find or forEach before writing the code.

Common Errors and Diagnosis

- Missing return in map.
- Using find when multiple results are required.
- Expecting filter to mutate the source array.
- Using forEach when a transformed array is needed.

Exit Check

290. map versus forEach?
291. filter versus find?
292. Which method would you use for a list of matching products?

Professional Habit

Save a working state, test it, and create a meaningful Git commit before leaving the class.

C2 - PRACTICE - Practice: Array Transformations and Mid-term Preparation

Why this matters: Use array methods to prepare data for the Mini Project 3 DOM application and strengthen problem-solving before the mid-term.

Learn This Before Coding

At this stage students should combine array transformations with DOM rendering. The important skill is not remembering method names in isolation, but selecting the correct operation from the requirement.

Key Ideas

- Read the requirement first.
- Choose the collection operation.

- Keep source data unchanged.
- Handle zero-result cases.
- Connect transformed data to rendering.

Reference Pattern

```
const result = products
  .filter(product => product.category === "book")
  .map(product => product.name);
```

Type the example yourself, run it, inspect the result, and then change at least one meaningful part. The goal is to understand the pattern rather than memorize the exact code.

Guided Laboratory

293. Transform a provided dataset.
294. Filter by one condition.
295. Find one record.
296. Render the result.
297. Test empty results.
298. Prepare one small DOM feature for the mid-term.

Independent Practice

299. Add search.
300. Combine two conditions.
301. Add a reset action.
302. Explain why your chosen array method is appropriate.

Challenge Task

Given a small product dataset and four UI requirements, choose the correct array method for each requirement and justify the choices.

Common Errors and Diagnosis

- Using the wrong method because its name sounds similar.
- Forgetting that filter can return an empty array.
- Writing mutation into a transformation.
- Ignoring the UI state when no items match.

Exit Check

303. Which array method returns a new array for every item?
304. Which returns the first match?
305. How do you handle no results?

Professional Habit

Save a working state, test it, and create a meaningful Git commit before leaving the class.

C3–C4 - SPECIAL 100-MINUTE EXAMINATION - Mid-term Practical Examination: 100 Minutes, 20 Marks

SPECIAL EXAMINATION BLOCK: Week 7 C3–C4 is one continuous 100-minute individual practical examination worth 20 marks; it replaces the normal C3 and C4 activities.

Why this matters: Demonstrate individual ability to build a small responsive interactive page using only material taught before the mid-term.

Learn This Before Coding

The mid-term is an individual practical examination. It occupies the entire Week 7 C3–C4 block. The scope is semantic HTML, responsive CSS, JavaScript interaction, DOM/events, array methods and debugging. React, Firebase and later-course topics are not required.

Key Ideas

- Work from the requirements first.
- Build functionality before polishing.
- Use semantic HTML.
- Make the page responsive.
- Implement at least one meaningful JavaScript interaction.
- Use DevTools when debugging.
- Test before submission.

Reference Pattern

Suggested examination workflow:

1. Read requirements.
2. Sketch the structure.
3. Build semantic HTML.
4. Add responsive CSS.
5. Add JavaScript interaction.
6. Test and debug.
7. Save and submit.

Type the example yourself, run it, inspect the result, and then change at least one meaningful part. The goal is to understand the pattern rather than memorize the exact code.

Guided Laboratory

306. Spend the first few minutes reading the full task.
307. Create the minimum semantic structure.
308. Implement the required layout.
309. Implement interaction.
310. Test at multiple widths.
311. Use Console/Elements/Network as appropriate.
312. Submit the requested repository or files.

Independent Practice

313. Before the examination, run at least one 100-minute mock build.
314. Practice prioritizing functionality over visual polishing.
315. Practice explaining your code without reading it line by line.

Challenge Task

Treat the examination as a build-and-debug problem rather than a memorization exercise. Do not spend most of the time polishing a feature that is not yet functional.

Common Errors and Diagnosis

- Poor time management.
- Over-polishing before functionality.
- Forgetting responsive testing.
- Using React or later topics when they are not required.
- Submitting without testing.

Exit Check

316. Can you build a small responsive interactive page within 100 minutes?
317. Can you explain your implementation choices?

Professional Habit

Save a working state, test it, and create a meaningful Git commit before leaving the class.

Week 8: Modern JavaScript, GitHub Workflow and Mini Project 3

C1 - CLASS / TEACHING - Modern JavaScript Essentials: Arrow Functions, Spread/Rest, Destructuring, Optional Chaining, Object Access, Iteration, Truthy/Falsy and Equality

Why this matters: Use modern JavaScript syntax that students will encounter frequently in React applications.

Learn This Before Coding

Arrow functions provide concise function syntax. Destructuring extracts values from arrays or objects. Spread syntax creates shallow copies or combines values. Rest syntax collects remaining arguments or properties. These features improve readability when used carefully.

Key Ideas

- Arrow functions are expressions and have different this behavior from traditional functions.
- Object destructuring extracts named properties.
- Spread copies array/object structure shallowly.
- Rest collects remaining values.
- Concise syntax should not be used at the expense of clarity.

Reference Pattern

```
const user = { name: "Amina", role: "student" };
const { name, role } = user;
const updated = { ...user, role: "developer" };
const add = (a, b) => a + b;

function sum(...numbers) {
  return numbers.reduce((total, n) => total + n, 0);
}
```

Type the example yourself, run it, inspect the result, and then change at least one meaningful part. The goal is to understand the pattern rather than memorize the exact code.

Guided Laboratory

318. Refactor simple functions to arrow functions.
319. Destructure an object.
320. Copy an array with spread.
321. Update an object immutably.
322. Use a rest parameter.

Independent Practice

323. Refactor a previous application without changing behavior.
324. Add an item immutably.
325. Use destructuring in function parameters.

326. Explain the difference between spread and rest.

Challenge Task

Refactor a working application using modern syntax while keeping the behavior unchanged.

Common Errors and Diagnosis

- Confusing spread and rest.
- Assuming spread creates a deep copy.
- Using overly compressed syntax.
- Destructuring a property that may not exist without understanding the result.

Exit Check

327. What does spread do?

328. What does destructuring do?

329. What does rest do?

Professional Habit

Save a working state, test it, and create a meaningful Git commit before leaving the class.

C2 - PRACTICE - Practice: Modern JavaScript Refactoring and Extensions

Why this matters: Use modern syntax and safer data access while practicing reasoning about JavaScript values and equality.

Learn This Before Coding

Optional chaining safely accesses nested properties when an intermediate value may be nullish. for-of iterates values in an iterable such as an array. for-in iterates enumerable property names. Truthy and falsy values influence conditions. Strict equality avoids type coercion and is normally preferred.

Key Ideas

- Use optional chaining when missing nested data is expected.
- Use for-of for array values.
- Use for-in for object keys when that is actually the desired operation.
- Know common falsy values.
- Prefer === unless there is a deliberate reason otherwise.

Reference Pattern

```
console.log(profile.user?.name);

for (const product of products) {
  console.log(product.name);
}

for (const key in profile) {
  console.log(key, profile[key]);
}

console.log(5 === "5"); // false
```

Type the example yourself, run it, inspect the result, and then change at least one meaningful part. The goal is to understand the pattern rather than memorize the exact code.

Guided Laboratory

- 330. Create nested objects with missing fields.
- 331. Use optional chaining.
- 332. Iterate array values with for-of.
- 333. Iterate object keys with for-in.
- 334. Test empty strings, zero, null and undefined.
- 335. Compare == and ===.

Independent Practice

- 336. Write defensive display functions.
- 337. Handle optional fields without crashing.
- 338. Explain why 0 should not automatically be treated as invalid.

Challenge Task

Create a data display function that remains stable when optional fields are missing and explain every conditional assumption.

Common Errors and Diagnosis

- Using for-in on arrays when values are intended.
- Treating 0 as missing data.
- Using optional chaining to hide required-data bugs.
- Assuming == and === behave identically.

Exit Check

- 339. for-of versus for-in?
- 340. What does ?. do?
- 341. Why is === normally preferred?

Professional Habit

Save a working state, test it, and create a meaningful Git commit before leaving the class.

C3 - CLASS / TEACHING - GitHub Workflow: Repositories, Branches and Pull Requests; Team Formation and Final Project Brief

Why this matters: Move from individual Git usage to a basic reviewable collaboration workflow.

Learn This Before Coding

A branch isolates a line of work. A pull request proposes a set of changes for review. The course intentionally keeps Git collaboration at repository, commit, branch and pull-request fundamentals. Advanced merge/conflict training is outside the core scope.

Key Ideas

- Repository stores the project.
- Branch isolates work.
- Commit records a coherent change.
- Push publishes branch commits.
- Pull request proposes changes for review.
- A review should focus on correctness, requirement satisfaction and maintainability.

Reference Pattern

```
git switch -c feature/course-card
git add .
git commit -m "Add course card"
git push -u origin feature/course-card
```

Type the example yourself, run it, inspect the result, and then change at least one meaningful part. The goal is to understand the pattern rather than memorize the exact code.

Guided Laboratory

342. Create a practice repository.
343. Create a feature branch.
344. Make a small feature.
345. Commit it.
346. Push the branch.
347. Open a pull request.
348. Review a partner's pull request.

Independent Practice

349. Write a useful review comment.
350. Verify changed files.
351. Use a clear branch name.
352. Create a final-project feature branch and draft a PR description.

Challenge Task

Write a pull request description that explains the requirement, implementation, test evidence and any known limitation.

Common Errors and Diagnosis

- Working directly on main.
- Huge unrelated pull requests.
- Vague commits.
- Reviewing personal style instead of actual defects or requirements.

Exit Check

353. Why use a branch?
354. What is a pull request for?
355. What makes a review comment useful?

Professional Habit

Save a working state, test it, and create a meaningful Git commit before leaving the class.

C4 - PRACTICE / EVALUATION - Mini Project 3 Evaluation: DOM Application or Debugging Task

PRACTICE / EVALUATION CLASS: Students apply C3 and combined weekly skills. On mini-project weeks, C4 is the dedicated individual evaluation with live verification.

Why this matters: Verify DOM logic, events, interaction, debugging and individual code ownership.

Learn This Before Coding

MP3 may be implemented as a DOM application or as a debugging task based on the course plan. The student must demonstrate a working artifact or corrected codebase and explain the relevant logic.

Key Ideas

- DOM selection and rendering should be understandable.
- Events should update behavior correctly.
- Data and UI flow should be explainable.
- Debugging should be evidence-driven.
- The student should be able to make a small change.

Reference Pattern

Required evidence:

- Working DOM application or corrected codebase
- Repository
- Meaningful commits
- Individual explanation

Type the example yourself, run it, inspect the result, and then change at least one meaningful part. The goal is to understand the pattern rather than memorize the exact code.

Guided Laboratory

356. Demonstrate the required feature.
357. Answer ownership/debugging questions.
358. Show the relevant DOM node or failing evidence.
359. Make one small live change if requested.
360. Record the rubric result.

Independent Practice

361. Practice explaining one bug and its root cause.
362. Practice showing the relevant DOM node in DevTools.
363. Practice rebuilding one small feature without copying.

Challenge Task

Do not prepare a presentation-heavy submission. Prepare a working artifact that you can explain, modify and debug.

Common Errors and Diagnosis

- Hard-coded repeated UI.
- Event handlers that do not update the intended data.
- Student cannot modify the project.
- Only screenshots are available.

Exit Check

364. Can you explain the event-to-UI flow?
365. What was the root cause of one bug?

Professional Habit

Save a working state, test it, and create a meaningful Git commit before leaving the class.

Week 9: APIs, Asynchronous JavaScript and Final Project Planning

C1 - CLASS / TEACHING - HTTP/HTTPS, APIs and Asynchronous JavaScript: Promises, async/await and try/catch/finally

Why this matters: Understand remote data and why browser applications need asynchronous programming.

Learn This Before Coding

An API endpoint exposes a service or resource. HTTP methods describe common operations. HTTPS protects HTTP traffic with TLS. Network operations are asynchronous because the browser should remain responsive while waiting for external work to complete.

Key Ideas

- Endpoint: address where a resource or service can be accessed.
- Request: information sent by the client.
- Response: information returned by the server.
- Asynchronous operation: work that completes later.
- UI state must account for waiting.

Reference Pattern

GET <https://jsonplaceholder.typicode.com/posts>

Type the example yourself, run it, inspect the result, and then change at least one meaningful part. The goal is to understand the pattern rather than memorize the exact code.

Guided Laboratory

366. Send a GET request.
367. Inspect status and JSON.
368. Change the endpoint or query.
369. Observe that network work completes after the request begins.
370. Write the state sequence idle → loading → success/error.

Independent Practice

371. Explain the endpoint.
372. Record a success and failure status.
373. Draw the API request lifecycle.
374. Explain why the browser should not freeze while waiting.

Challenge Task

Explain why a UI should never assume remote data is instantly available.

Common Errors and Diagnosis

- Ignoring response status.
- Treating remote data like a local variable.
- No loading state.
- Assuming HTTPS changes the API's application-level data structure.

Exit Check

375. What is an endpoint?
376. Why is network work asynchronous?
377. What information does a response provide?

Professional Habit

Save a working state, test it, and create a meaningful Git commit before leaving the class.

C2 - PRACTICE - Practice: Promises, async/await and Error Handling

Why this matters: Handle asynchronous results predictably and make failures visible.

Learn This Before Coding

Promises represent future results and have pending, fulfilled and rejected states. async functions return promises. await pauses the async function until the promise settles. try/catch handles failure, while finally is useful for cleanup such as removing a loading indicator.

Key Ideas

- A promise is not the final value itself.
- await works inside an async function.
- fetch does not automatically reject for every HTTP error.
- Check response.ok.
- Catch errors where the application can make a useful decision.
- Use finally for cleanup that must happen on both success and failure.

Reference Pattern

```
async function loadPosts() {  
  try {  
    const response = await fetch(url);  
    if (!response.ok) {  
      throw new Error(`HTTP ${response.status}`);  
    }  
    return await response.json();  
  } catch (error) {  
    console.error(error);  
    throw error;  
  } finally {  
    console.log("Finished");  
  }  
}
```

Type the example yourself, run it, inspect the result, and then change at least one meaningful part. The goal is to understand the pattern rather than memorize the exact code.

Guided Laboratory

378. Write a promise example.
379. Convert then/catch to async/await.
380. Add response.ok checking.
381. Add try/catch/finally.
382. Show loading while waiting.
383. Remove loading in finally.

Independent Practice

384. Create success and failure paths.
385. Create a reusable async function that either returns valid data or throws an error.
386. Test a deliberately broken URL.

Challenge Task

Build an async function whose caller can clearly distinguish successful data from failure.

Common Errors and Diagnosis

- Assuming fetch rejects on 404.
- Forgetting response.json().
- Catching and hiding the error.
- Forgetting to clear loading state.

Exit Check

387. Does fetch reject on a 404?
388. What does await do?
389. Why is response.ok important?

Professional Habit

Save a working state, test it, and create a meaningful Git commit before leaving the class.

C3 - CLASS / TEACHING - JSON, fetch, Dynamic Rendering, Loading/Error/Empty States, Network Tab and Final Project Proposal Workflow; SDLC/Agile Planning

Why this matters: Build a robust API screen that explicitly represents every important UI state.

Learn This Before Coding

JSON is structured text commonly used for API data. response.json() parses the response body asynchronously. A reliable interface distinguishes initial state, loading, success with data, success with no data, and error. The Network tab provides evidence about request URL, method, status and response.

Key Ideas

- JSON represents structured data.
- fetch starts the network request.
- response.ok should be checked.
- Loading is not the same as empty.
- Empty is not the same as error.
- Network evidence should guide debugging.

Reference Pattern

```
const response = await fetch(url);
if (!response.ok) throw new Error("Request failed");
const data = await response.json();
```

Type the example yourself, run it, inspect the result, and then change at least one meaningful part. The goal is to understand the pattern rather than memorize the exact code.

Guided Laboratory

390. Fetch users.
391. Render cards.
392. Add loading state.
393. Add error state.
394. Add empty state.
395. Inspect the Network tab.
396. Break the endpoint intentionally and repair it.

Independent Practice

- 397. Add retry.
- 398. Add result count.
- 399. Add search/filter.
- 400. Create a page that never leaves a blank UI during loading or failure.

Challenge Task

Design the UI states on paper before implementing the API request.

Common Errors and Diagnosis

- No loading state.
- Rendering before data exists.
- Ignoring Network.
- Treating an empty response as an error.

Exit Check

- 401. Why is empty not the same as error?
- 402. Why inspect Network?
- 403. What states should an API screen represent?

Professional Habit

Save a working state, test it, and create a meaningful Git commit before leaving the class.

C4 - PRACTICE / EVALUATION - Practice/Evaluation: Final Group Project Proposal Review, Scope Approval and SDLC/Agile Planning

Why this matters: Turn a project idea into a small, implementable final-project plan with clear users, requirements, backlog and acceptance criteria.

Learn This Before Coding

A user story describes a user's need. A backlog contains prioritized work. A sprint is a short planning window. Acceptance criteria define what must be true for a requirement to be considered complete. The course uses a practical SDLC sequence: understand problem → plan → implement → test/review → improve.

Key Ideas

- Problem before technology.
- Target users should be identifiable.
- Features should be prioritized.
- User stories should describe user value.
- Acceptance criteria should be testable.
- Scope must be controlled.

Reference Pattern

User story:

As a student, I want to view my courses so that I can track my workload.

Acceptance criteria:

- Authenticated user can open the page.
- Courses are displayed.
- Empty state is shown when there are none.
- Data comes from the required collection.

Type the example yourself, run it, inspect the result, and then change at least one meaningful part. The goal is to understand the pattern rather than memorize the exact code.

Guided Laboratory

- 404. Present the problem and target users.
- 405. Write 3–5 user stories.
- 406. Create a prioritized backlog.
- 407. Choose the first sprint.
- 408. Write acceptance criteria.
- 409. Identify one technical risk.
- 410. Obtain instructor scope approval.

Independent Practice

- 411. Remove one unnecessary feature.
- 412. Choose one feature that can be delivered end-to-end.
- 413. Write a project brief another developer could implement without a verbal explanation.

Challenge Task

Create a project brief containing problem, users, minimum features, routes, authentication need, Firestore use, acceptance criteria and development sequence.

Common Errors and Diagnosis

- Feature overload.
- No acceptance criteria.
- Technology chosen before the problem.
- Requirements too vague to test.

Exit Check

- 414. What is a user story?
- 415. Why must scope be controlled?
- 416. What makes acceptance criteria useful?

Professional Habit

Save a working state, test it, and create a meaningful Git commit before leaving the class.

Week 10: API Interfaces, Search, Persistence and Mini Project 4

C1 - CLASS / TEACHING - API-Driven Front-End Design: Request Flow, Dynamic Rendering, Loading/Error/Empty States, Search and Filter Interaction

Why this matters: Build the complete API page pattern by treating remote interaction as a state machine.

Learn This Before Coding

An API-driven interface moves through states. The application begins in an initial state, enters loading when a request starts, then reaches success with data, success with no data, or error. Keeping those states explicit prevents blank or misleading screens.

Key Ideas

- Initial, loading, success, empty and error are distinct states.
- Request code should not be mixed into every rendering branch.
- Data should be stored separately from status.
- Retry is a user action that starts another request.
- Old data should be handled deliberately when a new request fails.

Reference Pattern

```

async function loadProducts() {
  setStatus("loading");
  try {
    const response = await fetch(API_URL);
    if (!response.ok) throw new Error("Request failed");
    const data = await response.json();
    setData(data);
    setStatus(data.length ? "success" : "empty");
  } catch (error) {
    setError(error);
    setStatus("error");
  }
}

```

Type the example yourself, run it, inspect the result, and then change at least one meaningful part. The goal is to understand the pattern rather than memorize the exact code.

Guided Laboratory

417. Build the request function.
418. Create state variables.
419. Render each state.
420. Add retry.
421. Test empty response.
422. Test a broken endpoint.

Independent Practice

423. Add result count.
424. Add search.
425. Add filter.
426. Draw a UI state diagram before implementing the screen.

Challenge Task

Explain why an explicit state model is safer than checking whether an array happens to exist.

Common Errors and Diagnosis

- Loading never ends.
- Old data remains after an error without explanation.
- Empty treated as failure.
- Status not updated on every path.

Exit Check

427. What is the difference between empty and error?
428. Why use an explicit state?
429. Which state is shown while waiting?

Professional Habit

Save a working state, test it, and create a meaningful Git commit before leaving the class.

C2 - PRACTICE - Practice: API-Driven UI, Search, Filter and UI States

Why this matters: Practice the API state machine until students can build it without relying on a copied template.

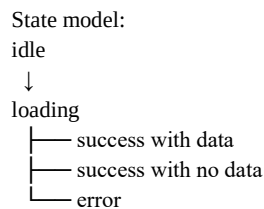
Learn This Before Coding

The task is to build a small API screen from a provided endpoint and requirements. Students should implement request handling, state transitions, rendering and debugging in a controlled sequence.

Key Ideas

- Write the state model first.
- Implement the request.
- Handle success.
- Handle empty.
- Handle error.
- Handle loading.
- Test all paths.

Reference Pattern



Type the example yourself, run it, inspect the result, and then change at least one meaningful part. The goal is to understand the pattern rather than memorize the exact code.

Guided Laboratory

430. Choose the provided API.
431. Create the state variables.
432. Implement the request.
433. Render success.
434. Add loading.
435. Add empty.
436. Add error and retry.

Independent Practice

437. Add search/filter after the state model works.
438. Break the API and repair it.
439. Have a partner test the empty and error paths.

Challenge Task

Create a test table showing the input condition, expected UI state, observed state and result.

Common Errors and Diagnosis

- Skipping one state.

- Using blank screen as loading.
- Not testing the error path.
- Adding search before the base API flow works.

Exit Check

440. Can you demonstrate all API states?
441. What evidence proves your error state works?

Professional Habit

Save a working state, test it, and create a meaningful Git commit before leaving the class.

C3 - CLASS / TEACHING - localStorage: Primitive Values, Objects and Simple Cart/Persistence Pattern

Why this matters: Persist small client-side data across page reloads using localStorage.

Learn This Before Coding

localStorage stores strings. Objects and arrays must therefore be serialized with JSON.stringify before storage and parsed with JSON.parse after retrieval. getItem can return null when the key does not exist, so initialization must handle missing data.

Key Ideas

- localStorage is browser-side persistence.
- Values are stored as strings.
- JSON.stringify converts objects/arrays to strings.
- JSON.parse converts strings back to JavaScript values.
- Missing keys must be handled.
- Do not treat localStorage as secure storage for sensitive information.

Reference Pattern

```
const saved = localStorage.getItem("cart");
const cart = saved ? JSON.parse(saved) : [];
```

```
localStorage.setItem("cart", JSON.stringify(cart));
```

Type the example yourself, run it, inspect the result, and then change at least one meaningful part. The goal is to understand the pattern rather than memorize the exact code.

Guided Laboratory

442. Save a theme preference.
443. Save a cart array.
444. Load it on startup.
445. Clear storage.
446. Inspect the Application panel.

Independent Practice

447. Build add/remove cart behavior.
448. Persist after each change.
449. Handle empty storage.
450. Add clear-cart.

Challenge Task

Refresh repeatedly and prove that the cart survives while explaining exactly what is stored as a string.

Common Errors and Diagnosis

- Calling methods on null.
- Storing objects directly.
- Parsing invalid JSON.
- Treating localStorage as secure sensitive storage.

Exit Check

- 451. Why stringify?
- 452. What happens when a key is missing?
- 453. Why is localStorage not suitable for sensitive data?

Professional Habit

Save a working state, test it, and create a meaningful Git commit before leaving the class.

C4 - PRACTICE / EVALUATION - Mini Project 4 Evaluation: API Application + Local Persistence + Deployment

PRACTICE / EVALUATION CLASS: Students apply C3 and combined weekly skills. On mini-project weeks, C4 is the dedicated individual evaluation with live verification.

Why this matters: Verify the complete API-driven front-end workflow: remote data, explicit UI states, search/filter where required, persistence and deployment.

Learn This Before Coding

MP4 requires an API application with local persistence, loading/error handling and deployment. The student should demonstrate the working flow and explain how data moves from the API into the UI and storage.

Key Ideas

- API data must be handled asynchronously.
- Loading/error/empty states must be visible.
- Persistence must survive refresh.
- The application should be responsive.
- The deployed application must work.

Reference Pattern

Minimum evidence:

- API data
- Loading state
- Error state
- Empty/no-result state
- Search/filter
- localStorage
- Responsive UI
- Deployment
- Repository

Type the example yourself, run it, inspect the result, and then change at least one meaningful part. The goal is to understand the pattern rather than memorize the exact code.

Guided Laboratory

- 454. Demonstrate the API request.
- 455. Demonstrate failure handling.
- 456. Refresh and demonstrate persistence.

- 457. Explain Network debugging.
- 458. Make one small live modification if possible.
- 459. Record the rubric result.

Independent Practice

- 460. Break the endpoint and repair it.
- 461. Show stored data in the Application panel.
- 462. Explain the API → state → UI → persistence flow.

Challenge Task

Explain every stage from remote request to visible UI and local persistence.

Common Errors and Diagnosis

- No error state.
- Persistence lost after refresh.
- Broken deployed asset paths.
- Student cannot explain state transitions.

Exit Check

- 463. Can you explain every stage of the API → UI → persistence flow?
- 464. How did you verify the deployed version?

Professional Habit

Save a working state, test it, and create a meaningful Git commit before leaving the class.

Week 11: Vite, React, Components, Props, State and Effects

C1 - CLASS / TEACHING - Vite Scaffolding, React Components, JSX, Props, Conditional Rendering and Lists

Why this matters: Move from page-level DOM manipulation to component-based UI development with React.

Learn This Before Coding

React components are reusable UI units. JSX allows JavaScript code to describe UI structure. Vite provides the development workflow. Instead of building one large page, students should think in terms of a component tree and clear responsibilities.

Key Ideas

- A component receives inputs and returns UI.
- JSX is syntax used to describe UI.
- Components should have focused responsibilities.
- Lists should use stable keys.
- The component tree should reflect the UI structure.

Reference Pattern

```
function CourseCard({ title, code }) {  
  return (  
    <article>  
      <h2>{title}</h2>  
    </article>  
  )  
}
```

```
<p>{code}</p>
</article>
);
}
```

Type the example yourself, run it, inspect the result, and then change at least one meaningful part. The goal is to understand the pattern rather than memorize the exact code.

Guided Laboratory

465. Create a Vite React project.
466. Create Header, Card and Footer.
467. Render a list.
468. Use stable keys.
469. Add an empty state.
470. Inspect the component structure.

Independent Practice

471. Create three cards.
472. Pass different data.
473. Create a reusable button.
474. Decompose an existing static page into a sensible component tree.

Challenge Task

Take a large static page and explain where you would split it into components and why.

Common Errors and Diagnosis

- Huge App component.
- Missing keys.
- Duplicated markup.
- Creating components with no meaningful responsibility.

Exit Check

475. What is a component?
476. Why are keys needed for lists?
477. What does JSX provide?

Professional Habit

Save a working state, test it, and create a meaningful Git commit before leaving the class.

C2 - PRACTICE - Practice: React Components, Props, Lists and Conditional UI

Why this matters: Make components configurable and data-driven through props, lists and conditional rendering.

Learn This Before Coding

Props are read-only inputs supplied by a parent component. Lists are usually rendered with map and stable keys. Conditional rendering allows a component to display different UI depending on data.

Key Ideas

- Props are inputs, not component-owned mutable state.
- Use map for repeated UI.

- Use stable keys.
- Use conditions to represent data state.
- Avoid unreadable nested ternaries.

Reference Pattern

```
function ProductCard({ product }) {
  return (
    <article>
      <h2>{product.name}</h2>
      {product.inStock
        ? <p>Available</p>
        : <p>Out of stock</p>}
    </article>
  );
}
```

Type the example yourself, run it, inspect the result, and then change at least one meaningful part. The goal is to understand the pattern rather than memorize the exact code.

Guided Laboratory

478. Build ProductCard.
479. Pass product objects.
480. Render an array.
481. Add conditional stock state.
482. Add an empty list message.

Independent Practice

483. Add category badges.
484. Create a reusable card.
485. Use the same component for many data objects without editing its internal structure.

Challenge Task

Build one component that can display many records only by changing its props.

Common Errors and Diagnosis

- Mutating props.
- Using index keys when stable ids exist.
- Nested ternaries that are difficult to read.
- Hard-coding data inside the component.

Exit Check

486. What are props?
487. Why are stable keys useful?
488. How does conditional rendering work?

Professional Habit

Save a working state, test it, and create a meaningful Git commit before leaving the class.

C3 - CLASS / TEACHING - Event Handling and useState; useEffect, API Calls with State and Practical React Project Structure

Why this matters: Model changing interface data through React state.

Learn This Before Coding

State is data owned by a component that changes over time and affects rendering. `useState` returns the current value and a setter. When state changes, React schedules a new render. Students should distinguish state from values that can be derived from existing state.

Key Ideas

- State changes over time.
- `useState` returns value and setter.
- Do not mutate state directly.
- Avoid storing values that can be calculated from existing state.
- Functional updates are useful when the next value depends on the previous value.

Reference Pattern

```
const [count, setCount] = useState(0);

<button onClick={() => setCount(count + 1)}>
  Count: {count}
</button>
```

Type the example yourself, run it, inspect the result, and then change at least one meaningful part. The goal is to understand the pattern rather than memorize the exact code.

Guided Laboratory

489. Build a counter.
490. Build a toggle.
491. Build selected-item interaction.
492. Inspect state behavior.
493. Explain which values are state and which are derived.

Independent Practice

494. Build cart count.
495. Add/remove items.
496. Display a derived total.
497. Use a functional state update for repeated updates.

Challenge Task

Given an interface, identify the minimum state required and the values that should be derived.

Common Errors and Diagnosis

- Direct mutation.
- Duplicated state.
- Stale state in multiple updates.
- Storing a derived total unnecessarily.

Exit Check

498. What does `useState` return?
499. What is derived data?
500. Why should state not be mutated directly?

Professional Habit

Save a working state, test it, and create a meaningful Git commit before leaving the class.

C4 - PRACTICE - Practice: React State and Effect Workflow

Why this matters: Use effects for external synchronization such as API requests while keeping loading and error state explicit.

Learn This Before Coding

Rendering should be predictable. Fetching remote data is an external side effect because it communicates with a system outside React's rendering calculation. `useEffect` is used to synchronize the component with that external system. Data, loading and error state should be represented explicitly.

Key Ideas

- Effects synchronize with external systems.
- Do not place ordinary calculations in effects unnecessarily.
- Use a dependency array intentionally.
- Store loading and error state.
- Check `response.ok`.
- Avoid infinite effect loops.

Reference Pattern

```
useEffect(() => {  
  async function loadUsers() {  
    const response = await fetch(API_URL);  
    if (!response.ok) throw new Error("Request failed");  
    const data = await response.json();  
    setUsers(data);  
  }  
  
  loadUsers();  
}, []);
```

Type the example yourself, run it, inspect the result, and then change at least one meaningful part. The goal is to understand the pattern rather than memorize the exact code.

Guided Laboratory

501. Fetch data in React.
502. Store loading and error.
503. Render results.
504. Inspect Network.
505. Explain why the effect runs when the dependency array is empty.

Independent Practice

506. Add retry.
507. Handle empty data.
508. Add search.
509. Create an effect that does not produce an infinite loop.

Challenge Task

Explain what external system the effect synchronizes with and why the dependency array has its chosen value.

Common Errors and Diagnosis

- Wrong dependency array.

- No response.ok.
- Async function confusion.
- Updating a dependency inside the effect and causing a loop.

Exit Check

510. What is a side effect?
511. Why is [] used here?
512. When should an effect run again?

Professional Habit

Save a working state, test it, and create a meaningful Git commit before leaving the class.

Week 12: Immutable State, React Router and ShadCN/ui

C1 - CLASS / TEACHING - Immutable State Updates, Lifting State Up and React Router: Routes, Outlet, Link and NavLink

Why this matters: Keep one reliable source of truth and update arrays and objects without mutation.

Learn This Before Coding

React state should be replaced rather than directly mutated. When sibling components need shared changing data, move the state to their nearest common parent and pass values and handlers through props. This creates one source of truth.

Key Ideas

- Replace arrays instead of mutating them.
- Replace objects instead of mutating their properties.
- Use functional updates when based on previous state.
- Lift shared state to the nearest common parent.
- Pass state down and events up.

Reference Pattern

```
setItems(previous =>
  previous.map(item =>
    item.id === id
      ? { ...item, quantity: item.quantity + 1 }
      : item
  )
);

setItems(previous =>
  previous.filter(item => item.id !== id)
);
```

Type the example yourself, run it, inspect the result, and then change at least one meaningful part. The goal is to understand the pattern rather than memorize the exact code.

Guided Laboratory

513. Build a parent/child selection example.
514. Lift selected state.

- 515. Pass the value and handler.
- 516. Update an array immutably.
- 517. Display the shared value in a sibling.

Independent Practice

- 518. Build a cart.
- 519. Update quantity.
- 520. Remove an item.
- 521. Display a count in a sibling component.

Challenge Task

Draw the component tree and mark exactly where each state value should be owned.

Common Errors and Diagnosis

- Duplicated state.
- Mutating the previous array.
- Unstable ids.
- Lifting state too high without need.

Exit Check

- 522. What is lifting state?
- 523. Why are immutable updates important?
- 524. Where should shared state normally live?

Professional Habit

Save a working state, test it, and create a meaningful Git commit before leaving the class.

C2 - PRACTICE - Practice: Immutable State, Lifting State and Basic Routing

Why this matters: Practice immutable updates and state ownership through a small multi-component application.

Learn This Before Coding

The practice task should make state ownership visible. Students should identify which component needs to know the data, move the state to the nearest common parent, and pass only the values and handlers required by children.

Key Ideas

- State ownership should be intentional.
- Use previous-state callbacks when appropriate.
- Use map/filter/spread for immutable updates.
- Keep derived values derived.
- Avoid duplicating the same source of truth.

Reference Pattern

```
function App() {  
  const [items, setItems] = useState([]);  
  
  function addItem(item) {  
    setItems(previous => [...previous, item]);  
  }  
}
```



```
}  
}
```

Type the example yourself, run it, inspect the result, and then change at least one meaningful part. The goal is to understand the pattern rather than memorize the exact code.

Guided Laboratory

- 525. Build a parent with two child components.
- 526. Move shared selection into the parent.
- 527. Pass the selected value down.
- 528. Pass an event handler down.
- 529. Update an array immutably.
- 530. Display a derived count.

Independent Practice

- 531. Add remove and update operations.
- 532. Explain why the state cannot remain in only one child.
- 533. Draw the data flow.

Challenge Task

Given a broken application with duplicated state, identify the single source of truth and refactor the state ownership.

Common Errors and Diagnosis

- Keeping duplicate state synchronized manually.
- Mutating nested objects.
- Passing unnecessary props.
- Putting shared state in an unrelated component.

Exit Check

- 534. Where is the source of truth?
- 535. How does a child request a state change?
- 536. How do you update nested data immutably?

Professional Habit

Save a working state, test it, and create a meaningful Git commit before leaving the class.

C3 - CLASS / TEACHING - Dynamic Routes, useParams, useNavigate, 404/Loading States and ShadCN/ui: Button, Card, Dialog and Table

Why this matters: Build realistic multi-view React applications using client-side routing.

Learn This Before Coding

Routes map URL paths to components. Link and NavLink provide client-side navigation. Outlet renders a matched child route inside a layout. Dynamic routes contain variable segments. useParams reads those values. useNavigate changes routes programmatically. Detail pages should account for loading and missing data.

Key Ideas

- Routes define URL-to-component relationships.
- Outlet renders nested child routes.
- NavLink can expose active navigation state.

- Dynamic segments capture identifiers.
- useParams reads route parameters.
- useNavigate performs programmatic navigation.
- A 404 route handles unmatched paths.

Reference Pattern

```
<Routes>
  <Route element={<Layout />}>
    <Route path="/" element={<Home />} />
    <Route path="/courses" element={<Courses />} />
    <Route path="/courses/:id" element={<CourseDetails />} />
    <Route path="*" element={<NotFound />} />
  </Route>
</Routes>
```

Type the example yourself, run it, inspect the result, and then change at least one meaningful part. The goal is to understand the pattern rather than memorize the exact code.

Guided Laboratory

537. Create Home, Courses and About routes.
538. Create a Layout.
539. Add NavLink.
540. Add Outlet.
541. Create a dynamic course route.
542. Read the id with useParams.
543. Navigate back with useNavigate.
544. Add a 404 route.

Independent Practice

545. Add a loading state to a detail page.
546. Add a not-found state.
547. Test direct URL navigation.
548. Draw the route tree before implementing it.

Challenge Task

Build one detail component that works for every course id without creating separate components for each course.

Common Errors and Diagnosis

- Missing Outlet.
- Wrong nested path.
- Parameter name mismatch.
- Comparing route ids with incompatible data types.
- No missing-data state.

Exit Check

549. What is Outlet?
550. Why use NavLink?
551. What does useParams provide?
552. When is useNavigate useful?

Professional Habit

Save a working state, test it, and create a meaningful Git commit before leaving the class.

C4 - PRACTICE - Practice: Routed React Pages Using ShadCN/ui Components

Why this matters: Use a practical component layer while keeping application state and data under the student's control.

Learn This Before Coding

Component libraries provide reusable UI primitives. The application still owns the data and state. A Dialog opens because application state says it should; the library supplies the presentation and interaction primitives. The course uses a practical subset of ShadCN/ui rather than attempting to cover the entire library.

Key Ideas

- Button represents an action.
- Card provides reusable content structure.
- Dialog supports focused interaction or confirmation.
- Table presents tabular data.
- Application state decides when a component should appear.
- Do not treat a UI library as a replacement for application logic.

Reference Pattern

```
<Card>
  <CardHeader>
    <CardTitle>CSE-2340</CardTitle>
  </CardHeader>
  <CardContent>
    <p>Software Development 1</p>
  </CardContent>
</Card>
```

Type the example yourself, run it, inspect the result, and then change at least one meaningful part. The goal is to understand the pattern rather than memorize the exact code.

Guided Laboratory

553. Use Button.
554. Build a Card.
555. Open a Dialog from state.
556. Render a Table from data.
557. Build a small dashboard using the four required component types.

Independent Practice

558. Add a confirmation dialog.
559. Add a disabled button state.
560. Customize a card.
561. Render a table from an array.

Challenge Task

Explain which part belongs to ShadCN/ui and which part belongs to your application's data/state logic.

Common Errors and Diagnosis

- Treating the library as magic.
- Mixing unrelated state.
- Duplicating component code.
- Using a table for non-tabular layout.

Exit Check

562. What belongs to app state?
563. Why does a UI library not replace application state?
564. When is a Dialog appropriate?

Professional Habit

Save a working state, test it, and create a meaningful Git commit before leaving the class.

Week 13: Forms, Reusable Components, Debugging and Mini Project 5

C1 - CLASS / TEACHING - Controlled React Forms, Practical Validation, Reusable Components, Props and Children

Why this matters: Make input state and visible form fields agree and validate user input clearly.

Learn This Before Coding

A controlled input receives its value from React state and updates that state through `onChange`. Validation should be explicit, readable and user-friendly. Each field needs a value, update mechanism, validation rule and useful error message.

Key Ideas

- Controlled input value comes from state.
- `onChange` updates the relevant state.
- Validation rules should be explicit.
- Invalid submission should not silently proceed.
- Error messages should identify the problem clearly.

Reference Pattern

```
const [form, setForm] = useState({
  name: "",
  email: ""
});

function handleChange(e) {
  const { name, value } = e.target;
  setForm(previous => ({
    ...previous,
    [name]: value
  }));
}
```

Type the example yourself, run it, inspect the result, and then change at least one meaningful part. The goal is to understand the pattern rather than memorize the exact code.

Guided Laboratory

- 565. Build a registration form.
- 566. Control each field.
- 567. Validate required inputs.
- 568. Prevent invalid submission.
- 569. Show errors.
- 570. Show a success state after valid submission.

Independent Practice

- 571. Add email validation.
- 572. Add minimum length.
- 573. Add reset.
- 574. Test empty, valid, invalid and boundary inputs.

Challenge Task

Create a validation table before coding: field, valid condition, invalid condition, message and submission behavior.

Common Errors and Diagnosis

- Controlled/uncontrolled warning.
- Validation only on submit when earlier feedback is required.
- Duplicated validation logic.
- Incorrect name attributes causing state updates to fail.

Exit Check

- 575. What makes an input controlled?
- 576. Where does its current value come from?
- 577. What should happen when validation fails?

Professional Habit

Save a working state, test it, and create a meaningful Git commit before leaving the class.

C2 - PRACTICE - Practice: Controlled Forms, Validation and Reusable Components

Why this matters: Turn the C1 form model into a complete practice application and test it systematically.

Learn This Before Coding

Practice should cover both the normal path and invalid paths. A good form is not only one that accepts valid input; it also explains what is wrong and prevents invalid state from being submitted.

Key Ideas

- Keep one clear form state object when appropriate.
- Use reusable change handling when fields share the same structure.
- Validate before committing the requested action.
- Show errors close to the relevant input.
- Test boundaries, not only ideal values.

Reference Pattern

```
function validate(form) {  
  const errors = {};
```

```

if (!form.name.trim()) {
  errors.name = "Name is required.";
}

if (!form.email.includes("@")) {
  errors.email = "Enter a valid email.";
}

return errors;
}

```

Type the example yourself, run it, inspect the result, and then change at least one meaningful part. The goal is to understand the pattern rather than memorize the exact code.

Guided Laboratory

578. Build the form.
579. Implement controlled inputs.
580. Write validation.
581. Display field errors.
582. Prevent invalid submission.
583. Display success.

Independent Practice

584. Add one boundary rule.
585. Reset the form.
586. Test with a partner.
587. Record at least five test cases and expected outcomes.

Challenge Task

Design and execute a small test matrix for your form before considering it complete.

Common Errors and Diagnosis

- Only testing valid input.
- Allowing submission despite errors.
- Unclear messages.
- Validation logic scattered across many handlers.

Exit Check

588. What test cases did you use?
589. How does the form prevent invalid submission?

Professional Habit

Save a working state, test it, and create a meaningful Git commit before leaving the class.

C3 - CLASS / TEACHING - React Debugging: Component Inspection and Common State/Prop Problems

Why this matters: Abstract repeated structure sensibly and debug React by tracing data flow through components.

Learn This Before Coding

Reusable components should have clear responsibilities and configurable inputs. `children` is useful when a wrapper controls layout or behavior while the caller controls inner content. React DevTools can inspect components, props and state. Common problems include wrong props, state owned in the wrong component, incorrect keys and effects that run unexpectedly.

Key Ideas

- Create components around repeated responsibility.
- Use props for configurable values.
- Use `children` for composable content.
- Avoid over-abstraction.
- Use React DevTools to inspect component state and props.
- Trace data flow before changing code.

Reference Pattern

```
function Panel({ title, children }) {
  return (
    <section>
      <h2>{title}</h2>
      <div>{children}</div>
    </section>
  );
}

function Product({ product, onSelect }) {
  return (
    <button onClick={() => onSelect(product.id)}>
      {product.name}
    </button>
  );
}
```

Type the example yourself, run it, inspect the result, and then change at least one meaningful part. The goal is to understand the pattern rather than memorize the exact code.

Guided Laboratory

590. Extract a repeated Card.
591. Create a Panel with children.
592. Pass props.
593. Compose components.
594. Inspect component tree.
595. Break one prop name intentionally.
596. Repair it with DevTools.

Independent Practice

597. Diagnose wrong list data.
598. Diagnose stale state.
599. Find a state ownership problem.
600. Given a broken project, identify the bug category before editing it.

Challenge Task

Before changing a React component, identify whether the failure is in data, props, state ownership, rendering, event handling or effects.

Common Errors and Diagnosis

- Changing many components at once.
- Only looking at the final screen.
- Excessive console logging without a hypothesis.
- Creating components that have many unrelated responsibilities.

Exit Check

601. What should you trace first from a wrong screen?
602. How can React DevTools help?
603. When should a component be extracted?

Professional Habit

Save a working state, test it, and create a meaningful Git commit before leaving the class.

C4 - PRACTICE / EVALUATION - Mini Project 5 Evaluation: React SPA with Routing, Reusable Components and ShadCN/ui, Deployed

PRACTICE / EVALUATION CLASS: Students apply C3 and combined weekly skills. On mini-project weeks, C4 is the dedicated individual evaluation with live verification.

Why this matters: Verify the major React competency through an individual demonstration of a deployed single-page application.

Learn This Before Coding

MP5 requires a deployed React SPA with routing, reusable components and ShadCN/ui. The student should demonstrate stateful interaction, explain component structure and show that the application works at different screen sizes.

Key Ideas

- Vite React project.
- Multiple routes.
- Reusable components.
- Stateful interaction.
- ShadCN/ui usage.
- Responsive UI.
- Deployment.
- Git history and individual explanation.

Reference Pattern

Evidence:

- Vite React project
- Routes
- Reusable components
- Stateful interaction
- ShadCN/ui
- Responsive UI
- Deployment
- Git history

Type the example yourself, run it, inspect the result, and then change at least one meaningful part. The goal is to understand the pattern rather than memorize the exact code.

Guided Laboratory

604. Demonstrate routes.
605. Demonstrate component reuse.

- 606. Demonstrate state.
- 607. Demonstrate ShadCN/ui.
- 608. Answer one debugging question.
- 609. Make a small live change if time permits.

Independent Practice

- 610. Use React DevTools during explanation.
- 611. Prepare to identify where state is owned.
- 612. Practice explaining which route renders which component.

Challenge Task

Do not hide behind screenshots. Demonstrate the running application and explain its data flow.

Common Errors and Diagnosis

- Everything inside App.
- Duplicated state.
- No stable keys.
- Direct deployment route failure.
- Student cannot explain a component.

Exit Check

- 613. Where is state owned?
- 614. Which route renders which component?
- 615. What ShadCN/ui feature did you use and why?

Professional Habit

Save a working state, test it, and create a meaningful Git commit before leaving the class.

Week 14: Firebase Authentication, Firestore and Mini Project 6

C1 - CLASS / TEACHING - Firebase Project, SDK and Authentication: Google Sign-in/Sign-out, Email/Password, Auth State and Private Routes Without Context API

Why this matters: Connect a React front end to practical identity management using Firebase Authentication.

Learn This Before Coding

Authentication answers who the user is. Authorization answers what the user is allowed to access. Firebase Authentication can provide sign-in, sign-out and observable authentication state. The first practical flow uses Google sign-in.

Key Ideas

- Authentication identifies a user.
- Authorization controls access.
- Provider configuration must be enabled.
- Auth state may not be known immediately on first render.
- Sign-out returns the application to an unauthenticated state.

Reference Pattern

```
const provider = new GoogleAuthProvider();
```

```
await signInWithPopup(auth, provider);
```

Type the example yourself, run it, inspect the result, and then change at least one meaningful part. The goal is to understand the pattern rather than memorize the exact code.

Guided Laboratory

- 616. Connect Firebase.
- 617. Enable Google provider.
- 618. Add sign-in.
- 619. Display user information.
- 620. Add sign-out.
- 621. Test signed-out and signed-in states.
- 622. Add an authentication loading state.

Independent Practice

- 623. Change navigation based on auth state.
- 624. Write the auth state transitions before implementing the UI.
- 625. Explain the difference between identity and permission.

Challenge Task

Draw the auth state sequence: unknown/loading → signed out or signed in → sign out → signed out.

Common Errors and Diagnosis

- Provider not enabled.
- Configuration mismatch.
- Assuming the user is known on first render.
- Showing protected content while auth state is still loading.

Exit Check

- 626. Authentication versus authorization?
- 627. Why is auth loading useful?
- 628. What happens after sign-out?

Professional Habit

Save a working state, test it, and create a meaningful Git commit before leaving the class.

C2 - PRACTICE - Practice: Firebase Authentication and Private Routes

Why this matters: Implement a practical email/password authentication flow and guard the front-end navigation without using Context API.

Learn This Before Coding

`createUserWithEmailAndPassword` and `signInWithEmailAndPassword` implement the basic email/password flow. `onAuthStateChanged` allows the application to respond to authentication changes. A private route is a UI/navigation guard, not database security. Firestore rules remain responsible for database authorization.

Key Ideas

- Registration creates an account.

- Login authenticates an existing account.
- `onAuthStateChanged` observes changes.
- Private routes control UI navigation.
- Auth loading prevents premature redirects.
- Database rules are still required for data protection.

Reference Pattern

```
onAuthStateChanged(auth, user => {
  console.log(user ? user.email : "signed out");
});
```

Type the example yourself, run it, inspect the result, and then change at least one meaningful part. The goal is to understand the pattern rather than memorize the exact code.

Guided Laboratory

629. Build registration.
630. Build login.
631. Track auth state.
632. Create a private route.
633. Test signed-out access.
634. Test wrong password.
635. Test logout.
636. Redirect to login.

Independent Practice

637. Add auth-loading UI.
638. Test redirect behavior.
639. Explain why hiding a button is not database security.

Challenge Task

Create a private route and explain exactly what it protects and what it does not protect.

Common Errors and Diagnosis

- No auth loading.
- Redirect loop.
- Assuming private route protects Firestore.
- Mixing authentication state with unrelated application state.

Exit Check

640. What does `onAuthStateChanged` do?
641. What does a private route protect?
642. Why are Firestore rules still necessary?

Professional Habit

Save a working state, test it, and create a meaningful Git commit before leaving the class.

C3 - CLASS / TEACHING - Firestore: One Collection, Basic Read/Write and Supplied Security-Rules Starter

Why this matters: Connect front-end state to one persistent Firestore collection while understanding the role of security rules.

Learn This Before Coding

Firestore stores documents inside collections. The SDK can create and read documents. Security rules decide whether requests are allowed. Authentication and authorization remain separate concepts. The course intentionally limits Firestore to one collection and basic read/write.

Key Ideas

- Collection contains documents.
- Document contains fields.
- The SDK performs data operations.
- Rules decide whether a request is allowed.
- Authenticated does not automatically mean authorized for every operation.

Reference Pattern

```
const ref = collection(db, "courses");

await addDoc(ref, {
  code: "CSE-2340",
  title: "Software Development 1"
});

const snapshot = await getDocs(ref);

const courses = snapshot.docs.map(d => ({
  id: d.id,
  ...d.data()
}));
```

Type the example yourself, run it, inspect the result, and then change at least one meaningful part. The goal is to understand the pattern rather than memorize the exact code.

Guided Laboratory

643. Create the collection.
644. Add a document.
645. Read documents.
646. Render the list/table.
647. Use the supplied rules starter.
648. Test permission denied.
649. Explain auth versus rules.

Independent Practice

650. Add a create form.
651. Show the new data.
652. Test signed-out access.
653. Draw React UI → Firebase SDK → rules → Firestore → response → state → UI.

Challenge Task

Explain why a successful UI login does not automatically guarantee permission to every Firestore operation.

Common Errors and Diagnosis

- Permission denied.
- Wrong collection name.

- No stable document id.
- Rules too open.
- Confusing authentication with authorization.

Exit Check

- 654. Collection versus document?
- 655. What do rules protect?
- 656. How does Firestore data reach the screen?

Professional Habit

Save a working state, test it, and create a meaningful Git commit before leaving the class.

C4 - PRACTICE / EVALUATION - Mini Project 6 Evaluation: Authentication + One Firestore Collection + Deployment

PRACTICE / EVALUATION CLASS: Students apply C3 and combined weekly skills. On mini-project weeks, C4 is the dedicated individual evaluation with live verification.

Why this matters: Verify the complete cloud-connected front-end flow through an individual demonstration.

Learn This Before Coding

MP6 focuses on authentication, private access, one Firestore collection, basic persistence, configuration safety and deployment. The student must explain the authentication-to-data flow and demonstrate it live.

Key Ideas

- Authentication works.
- Protected route works.
- One collection is used.
- Create/read behavior works.
- Rules are not unnecessarily open.
- Responsive UI is deployed.
- Student can explain configuration and security decisions.

Reference Pattern

Evidence:

- Login/logout
- Protected route
- One Firestore collection
- Create/read
- Responsive UI
- Deployed URL
- Safe rules/configuration

Type the example yourself, run it, inspect the result, and then change at least one meaningful part. The goal is to understand the pattern rather than memorize the exact code.

Guided Laboratory

- 657. Demonstrate login/logout.
- 658. Demonstrate private route.
- 659. Create/read a document.
- 660. Explain the security decision.
- 661. Show signed-out behavior.
- 662. Make one small live UI modification if possible.

Independent Practice

- 663. Explain authentication → protected UI → Firestore.
- 664. Test a permission-denied condition.
- 665. Review rules before evaluation.

Challenge Task

Explain the complete authentication → protected UI → Firestore flow without treating a private route as database security.

Common Errors and Diagnosis

- Rules too open.
- No auth loading.
- Configuration misunderstanding.
- Student cannot explain which part is authentication and which part is authorization.

Exit Check

- 666. Authentication versus authorization?
- 667. Why do Firestore rules matter?
- 668. What does the private route actually protect?

Professional Habit

Save a working state, test it, and create a meaningful Git commit before leaving the class.

Week 15: Project Polish, GitHub Review, Integration and Final Showcase

C1 - CLASS / TEACHING - Project Polish, Responsive Behavior, Accessibility Checks, Lighthouse Audit and GitHub Pull-Request Review Workflow

Why this matters: Turn a working application into a credible final product by checking functionality first, then responsiveness, accessibility and performance.

Learn This Before Coding

Quality work is iterative. Verify functionality before cosmetic polishing. Test multiple viewport sizes, keyboard navigation, headings, forms, images, focus behavior, Console output and Network requests. Lighthouse is diagnostic; a score is not a substitute for understanding the underlying issue.

Key Ideas

- Test 320–390px, tablet and desktop.
- Check keyboard navigation.
- Check headings/forms/images/focus.
- Check Console.
- Check Network.
- Run Lighthouse.
- Fix and rerun.

Reference Pattern

Final audit:

1. Test narrow mobile.
2. Test tablet.
3. Test desktop.
4. Keyboard navigation.
5. Console.
6. Network.
7. Lighthouse.
8. Fix and rerun.

Type the example yourself, run it, inspect the result, and then change at least one meaningful part. The goal is to understand the pattern rather than memorize the exact code.

Guided Laboratory

669. Test several widths.
670. Keyboard through controls.
671. Check headings, forms, images and focus.
672. Run Lighthouse.
673. Fix three issues.
674. Record before/after evidence.
675. Retest.

Independent Practice

676. Write a mini quality report: issue → evidence → fix → verification.
677. Ask a teammate to test one critical journey.
678. Fix one issue discovered by another person.

Challenge Task

Do not chase a score without understanding the problem it represents. Prioritize broken functionality, accessibility and meaningful performance issues.

Common Errors and Diagnosis

- Cosmetic fixes before broken features.
- Testing only one screen size.
- Chasing scores.
- Ignoring keyboard interaction.

Exit Check

679. What does Lighthouse tell you?
680. Which issue did you fix and how did you verify it?

Professional Habit

Save a working state, test it, and create a meaningful Git commit before leaving the class.

C2 - PRACTICE - Practice: Final Project Polish and Submission Preparation

Why this matters: Apply the Week 15 audit method directly to the final group project and produce evidence of improvement.

Learn This Before Coding

Students should now work like developers preparing a release. The goal is not to add major features. The goal is to find, fix and verify issues in the existing product.

Key Ideas

- Stabilize before adding features.
- Use evidence for every fix.
- Test the critical user journey.
- Check accessibility.
- Check responsiveness.
- Check Console and Network.
- Keep changes focused.

Reference Pattern

Issue record:
Problem:
Evidence:
Root cause:
Fix:
Verification:

Type the example yourself, run it, inspect the result, and then change at least one meaningful part. The goal is to understand the pattern rather than memorize the exact code.

Guided Laboratory

681. Run the final project at three viewport ranges.
682. Perform a keyboard pass.
683. Check Console.
684. Check Network.
685. Run Lighthouse.
686. Fix at least three issues.
687. Retest after each important fix.

Independent Practice

688. Create a release-readiness checklist.
689. Have a teammate reproduce one issue.
690. Document the issue and verification evidence.

Challenge Task

Produce a short quality report showing that each selected problem was observed, diagnosed, fixed and retested.

Common Errors and Diagnosis

- Adding unrelated features.
- Changing many files without testing.
- Fixing only the visible symptom.
- Failing to retest after a change.

Exit Check

691. Which issues did you fix?
692. What evidence proves the fixes work?

Professional Habit

Save a working state, test it, and create a meaningful Git commit before leaving the class.

C3 - CLASS / TEACHING - Integration Studio and Final Debugging

Why this matters: Review team work, stabilize integration and prepare the final repository without introducing advanced Git workflows outside the syllabus.

Learn This Before Coding

The final workflow uses repository, commit, branch and pull-request fundamentals. Advanced merge/conflict lessons are intentionally outside the core scope. Integration testing checks whether the complete user journey still works after changes.

Key Ideas

- Review requirements, not personal preferences.
- Check changed files.
- Run the application.
- Test the deployed build.
- Check Console.
- Check responsiveness.
- Verify README and repository history.
- Use smoke testing after integration.

Reference Pattern

Review checklist:
Requirement satisfied?
Changed files relevant?
No Console errors?
Responsive?
README updated?
Deployed build tested?

Type the example yourself, run it, inspect the result, and then change at least one meaningful part. The goal is to understand the pattern rather than memorize the exact code.

Guided Laboratory

693. Review a teammate PR.
694. Inspect changed files.
695. Run the project.
696. Leave a useful review comment.
697. Prepare the final repository.
698. Run the smoke test.
699. Fix the highest-impact integration issue.

Independent Practice

700. Verify each member has a branch.
701. Verify meaningful commits.
702. Verify end-to-end ownership.
703. Verify README and deployment.
704. Have a teammate perform the critical journey.

Challenge Task

Perform an evidence-based review without rewriting a teammate's code. Focus on requirement satisfaction, defects, test evidence and maintainability.

Common Errors and Diagnosis

- Treating style preference as a defect.
- Ignoring deployed behavior.
- Huge PR.
- Testing only one authentication state.
- Assuming localhost equals deployment.

Exit Check

705. What makes a useful review comment?

706. What is regression testing?

707. What should a smoke test cover?

Professional Habit

Save a working state, test it, and create a meaningful Git commit before leaving the class.

C4 - FINAL SHOWCASE - Final Group Project Showcase and Individual Viva

Why this matters: Demonstrate the final product, team integration and individual ownership through a live showcase and viva.

Learn This Before Coding

The final project integrates React, routing, ShadCN/ui, Firebase Authentication, Firestore and deployment. The project requires at least three routes, protected authentication, one Firestore collection, a meaningful ShadCN/ui feature, responsive deployment, README, individual end-to-end ownership, individual branches and pull requests, at least eight meaningful commits per member and an individual viva.

Key Ideas

- Demonstrate the problem and target users.
- Show the main user journey.
- Explain routes.
- Demonstrate authentication.
- Demonstrate Firestore.
- Explain ShadCN/ui usage.
- Show responsive behavior.
- Show deployment.
- Explain individual contribution.
- Answer viva questions.

Reference Pattern

Suggested demonstration:

1. Problem and users
2. Main user journey
3. Routes
4. Authentication
5. Firestore
6. ShadCN/ui
7. Responsive behavior

- 8. Deployment
- 9. Individual contribution
- 10. Viva

Type the example yourself, run it, inspect the result, and then change at least one meaningful part. The goal is to understand the pattern rather than memorize the exact code.

Guided Laboratory

- 708. Demonstrate the deployed application.
- 709. Show the repository.
- 710. Show branch and PR evidence.
- 711. Explain the individual feature.
- 712. Answer viva questions.
- 713. Make a small modification if requested.

Independent Practice

- 714. Be ready to modify your feature.
- 715. Explain one bug and its fix.
- 716. Explain one architecture decision.
- 717. Practice the complete user journey while signed in and signed out.

Challenge Task

Present behavior and decisions rather than reading long source-code listings. Individual ownership is demonstrated by the ability to explain, modify and debug the submitted work.

Common Errors and Diagnosis

- Member cannot explain the feature.
- Demo-only evidence.
- No repository history.
- Cannot explain authentication or Firestore flow.
- Final deployment differs from the demonstrated local version.

Exit Check

- 718. What problem does the project solve?
- 719. Where is state owned?
- 720. How does authentication protect the UI?
- 721. How does Firestore data reach the screen?

Professional Habit

Save a working state, test it, and create a meaningful Git commit before leaving the class.

Mini Projects and Final Project Guide

The six mini projects are individual and progressive. Each has a dedicated evaluation class. The final group project integrates the technologies taught throughout the course.

Project	Due / Evaluation	Marks	Deliverable	Evaluation Focus
MP1	Week 3 C4	5	Semantic HTML + CSS + responsive layout + deployment	Structure, styling, responsiveness, deployment and individual explanation
MP2	Week 5 C4	5	Own responsive design + Tailwind +	Design translation, Tailwind

			deployment	usage, responsive behavior, repository/deployment
MP3	Week 8 C4	7	DOM application or debugging task	DOM logic, events, interaction, debugging and code ownership
MP4	Week 10 C4	7	API application + localStorage + deployment	Fetch/API handling, UI states, persistence, debugging and deployment
MP5	Week 13 C4	8	React SPA + routing + reusable components + ShadCN/ui + deployment	Components, props/state, routing, UI library usage and deployment
MP6	Week 14 C4	8	Firebase Authentication + one Firestore collection + deployment	Authentication, private access, persistence, configuration safety and deployment

Common Mini-Project Rubric

Criterion	Typical Weight	Evidence
Core functionality	40%	Required feature works correctly.
Technical implementation	25%	Appropriate HTML, CSS, JavaScript, React or Firebase technique is used.
Responsiveness / usability	15%	Interface behaves appropriately at tested widths.
Code organization	10%	Readable structure and naming.
Individual demonstration	10%	Student can explain, modify or debug their own work.

Final Group Project - Development Blueprint

Teams of three are formed in Week 8. The project must integrate the course stack rather than introduce an unrelated technology stack.

- At least three routes.
- Authentication with protected routes.
- One Firestore collection with basic read/write.
- At least one meaningful ShadCN/ui feature.
- Responsive design and deployment.
- README with setup and feature information.
- Each member owns at least one feature end-to-end.
- Each member works on an individual branch and opens a pull request.
- Each member makes at least eight meaningful commits.
- Each member completes an individual viva.

Recommended Development Sequence

722. Define the problem and target users.
723. List minimum viable features.
724. Write acceptance criteria.
725. Create route and page requirements.
726. Create the GitHub repository and branch plan.
727. Build the responsive interface.
728. Convert repeated UI into React components.
729. Add routing.
730. Add authentication and protected UI.
731. Add one Firestore collection.

- 732. Integrate ShadCN/ui.
- 733. Test responsiveness and accessibility.
- 734. Review pull requests.
- 735. Deploy.
- 736. Run a smoke test.
- 737. Prepare the viva.

Suggested README Structure

```
# Project Name

## Problem Statement
## Users
## Features
## Technology
## Setup
## Routes
## Authentication
## Firestore
## Deployment
## Team Contributions
```

Mid-term Practical Examination - Announcement and Preparation

Item	Information
Format	Individual practical examination
Duration	100 minutes
Marks	20
Scope	Material taught before the mid-term
Expected areas	Semantic HTML, responsive CSS, JavaScript interaction, DOM/events, array methods and debugging
Not required	React, Firebase or later-course topics
Emphasis	Implementation, responsiveness, interaction and debugging

Week 7 C3–C4 is reserved for the 100-minute individual practical examination. The normal C3 and C4 activities are replaced by the examination.

Recommended Student Preparation

- 738. Practice semantic HTML from an empty file.
- 739. Practice responsive Flexbox and Grid layouts.
- 740. Practice JavaScript variables, arrays, objects and functions.
- 741. Practice DOM selection and manipulation.
- 742. Practice event handling and preventDefault.
- 743. Practice map, filter, find and forEach.
- 744. Practice debugging with Console and Elements.
- 745. Run at least one complete 100-minute mock build.

Professional Debugging and Engineering Habits

A capable front-end engineer does not solve every problem by changing code randomly. Observe the behavior, form a hypothesis, test it, and keep the smallest change that fixes the root cause.

Professional Debugging and Engineering Habits

A capable front-end engineer does not solve every problem by changing code randomly. Observe the behavior, form a hypothesis, test it, and keep the smallest change that fixes the root cause.

Ten-Step Debugging Loop

746. Reproduce the problem reliably.
747. State expected behavior.
748. State actual behavior.
749. Read the Console error carefully.
750. Inspect DOM and computed styles for structural or visual problems.
751. Inspect Network for APIs, assets and routes.
752. Inspect Application storage for persistence problems.
753. Reduce the problem to the smallest failing example.
754. Change one thing and retest.
755. Explain the root cause after fixing it.

Diagnostic Map

Symptom	Inspect First	Common Cause
Blank page	Console	Runtime or import error
CSS ignored	Styles and Computed	Selector, specificity or source order
Mobile overflow	Responsive and Computed	Fixed width, box model or grid issue
Button does nothing	Console and DOM	Wrong selector or event handler
Form reloads	Event handler	Missing preventDefault
API missing	Network	Wrong URL, status, parsing or state
Loading never ends	State and effect	State not updated on all paths
Data disappears	Application storage	Persistence not saved or parsed
React stale UI	React DevTools	State mutation or ownership
Route fails	Router and Network	Wrong path or deployment configuration
Firebase denied	Auth and rules	Unauthenticated request or rejected rule

Front-End Engineer Readiness Checklist

HTML

- Create valid HTML from scratch.
- Choose semantic elements by meaning.
- Create accessible forms.
- Create meaningful navigation.
- Use tables only for tabular information.

CSS

- Explain the box model.
- Build Flexbox layouts.
- Build Grid layouts.
- Write mobile-first responsive CSS.
- Diagnose specificity and cascade.
- Use positioning appropriately.
- Inspect computed styles.

JavaScript

- Use variables, arrays, objects and functions.
- Choose map, filter, find and forEach correctly.
- Use modern syntax.
- Reason about truthy/falsy and equality.
- Debug with Console.

DOM and Browser

- Select DOM nodes.
- Create and remove nodes.
- Handle events.
- Prevent default browser behavior when needed.
- Model data + actions + rendering.

APIs

- Explain request/response/status.
- Use fetch and async/await.
- Handle loading/error/empty.
- Inspect Network.
- Implement search/filter.
- Persist simple client state.

React

- Create a Vite project.
- Design a component tree.
- Pass props.
- Use state.
- Update arrays and objects immutably.
- Use effects appropriately.
- Debug with React DevTools.

Routing and UI

- Create routes.
- Use Outlet.
- Use dynamic routes and useParams.
- Use useNavigate.
- Use Button, Card, Dialog and Table from ShadCN/ui.

Firestore

- Authenticate.
- Observe auth state.
- Protect routes.
- Read/write one Firestore collection.
- Explain security rules.

Professional Workflow

- Use Git.
- Create branches.
- Open pull requests.
- Explain commits.
- Deploy.
- Debug systematically.

Appendix A - Essential Command Sheet

Purpose	Command
Node version	node --version
npm version	npm --version
Create Vite app	npm create vite@latest
Install packages	npm install
Start dev server	npm run dev
Git init	git init
Git status	git status
Stage	git add .
Commit	git commit -m "message"
Create branch	git switch -c feature/name
Push branch	git push -u origin feature/name
History	git log --oneline

Appendix B - Essential Patterns

```
// DOM
const button = document.querySelector("#saveButton");
button.addEventListener("click", () => console.log("Clicked"));
```

```
// Arrays
const active = products.filter(p => p.active);
const names = active.map(p => p.name);
```

```
// Async
async function loadData() {
  const response = await fetch(url);
  if (!response.ok) throw new Error("Request failed");
  return response.json();
}
```

```
// React state
const [count, setCount] = useState(0);
```



```
// Immutable update
setItems(previous => [...previous, newItem]);

// Route
<Route path="/courses/:id" element={<CourseDetails />} />;
```

Appendix C - Before Asking for Help

756. What were you trying to build?
757. What did you expect?
758. What actually happened?
759. What error or browser evidence did you observe?
760. What have you already tested?

Online Practice Sandboxes and Interactive Tools

These resources are supplementary practice environments. They support isolated experimentation and debugging; they do not replace the syllabus topics or the course repository workflow.

Resource	Primary use	Syllabus use	Link
CodePen	HTML/CSS/JavaScript experiments and reduced test cases	Weeks 1–6	Open resource
Flexbox Froggy	Interactive Flexbox alignment practice	Week 3 C1/C2	Open resource
Grid Garden	Interactive CSS Grid practice	Week 3 C3/C2 follow-up	Open resource
CSS Diner	CSS selector practice	Weeks 2–3	Open resource
Python Tutor JavaScript Visualizer	Step-by-step JavaScript execution visualization	Weeks 5–8	Open resource
React Learn	Official React learning/reference	Weeks 11–13	Open resource
Vite Guide	Official Vite setup and development guide	Week 11	Open resource
Hoppscotch	Browser-based API request testing	Weeks 9–10	Open resource
JSONPlaceholder	Practice REST API	Weeks 9–10	Open resource
Chrome DevTools	DOM, Console, Network, responsive and computed-style inspection	Throughout	Open resource
Figma	Design handoff, measurement and asset	Week 4	Open resource

	inspection		
Pixso	Design handoff and asset inspection	Week 4	<u>Open resource</u>

Professional question: Do not ask only, “Why is my code not working?” Give the smallest relevant code, expected behavior, actual behavior and evidence from DevTools.

FINAL DELIVERY NOTE: For every normal week, the intended rhythm is C1 teaching → C2 practice → C3 teaching → C4 practice/evaluation. The class labels in this manual explicitly identify that distinction.